第3章 Python简介

1. Python简介
2. Python开发环境
3. Python基础编程
4. 相关库介绍

Python起源

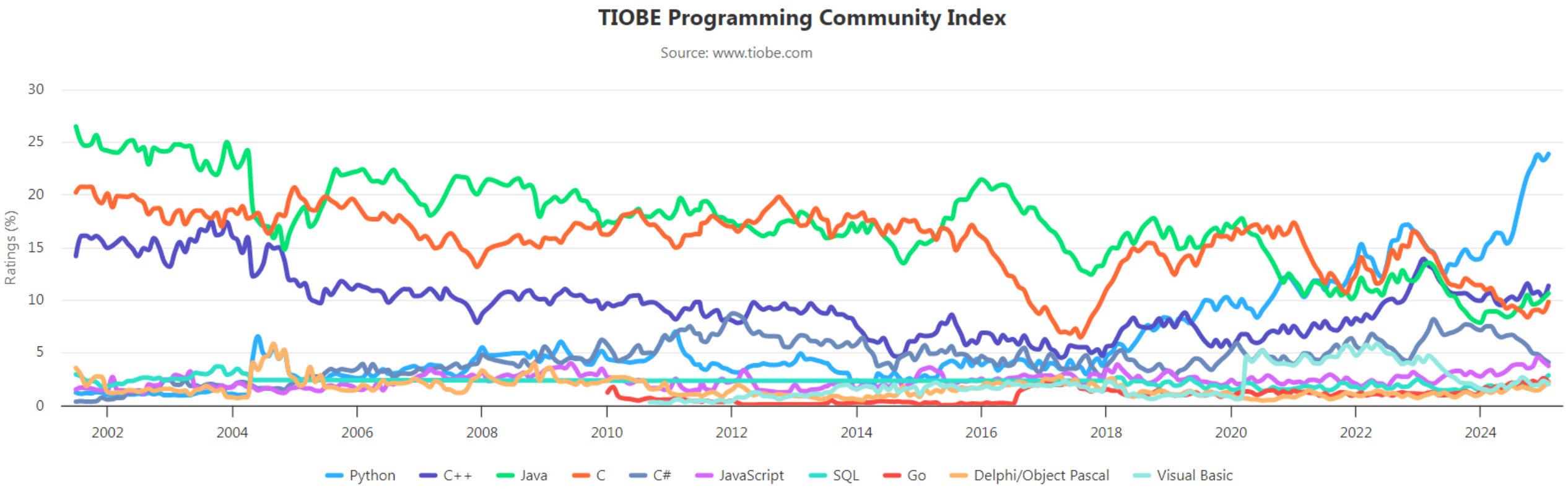
Guido van Rossum

(1956 -)

1989年末，Guido van Rossum为了打发圣诞节的无聊，创造了Python语言。2005年12月，入职Google工作。2012年12月，加入Dropbox公司

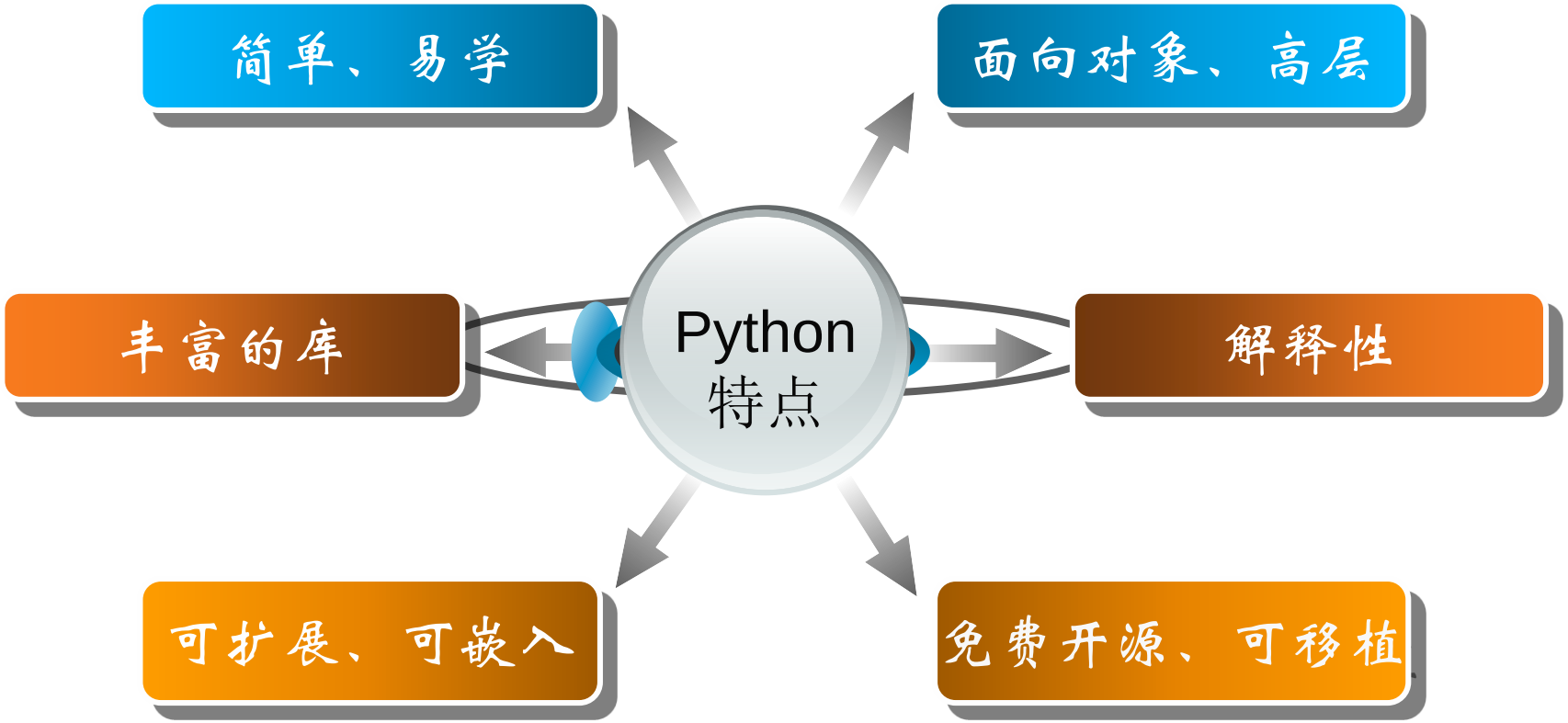
TIOBE Index for February 2025

Feb 2025	Feb 2024	Change	Programming Language	Ratings	Change
1	1		Python	23.88%	+8.72%
2	3	^	C++	11.37%	+0.84%
3	4	^	Java	10.66%	+1.79%
4	2	v	C	9.84%	-1.14%
5	5		C#	4.12%	-3.41%
6	6		JavaScript	3.78%	+0.61%
7	7		SQL	2.87%	+1.04%
8	8		Go	2.26%	+0.53%
9	12	^	Delphi/Object Pascal	2.18%	+0.78%
10	9	v	Visual Basic	2.04%	+0.52%


TIOBE Programming Community Index
Source: www.tiobe.com
Ratings (%)
30
25
20
15
10
5
0
2002
2004
2006
2008
2010
2012
2014
2016
2018
2020
2022
2024
Python
C++
Java
C
C#
JavaScript
SQL
Go
Delphi/Object Pascal
Visual Basic

Programming Language	2025	2020	2015	2010	2005	2000	1995	1990	1985
Python	1	3	7	7	7	24	23	-	-
C++	2	4	4	4	3	2	1	2	13
C	3	2	1	2	1	1	2	1	1
Java	4	1	2	1	2	3	-	-	-
C#	5	5	5	6	9	9	-	-	-
JavaScript	6	7	8	9	10	7	-	-	-
Go	7	16	36	184	-	-	-	-	-
Visual Basic	8	19	234	-	-	-	-	-	-
SQL	9	9	-	-	100	-	-	-	-
Fortran	10	30	31	25	15	18	5	8	11
PHP	13	8	6	3	5	29	-	-	-
Ada	25	35	30	26	16	17	7	4	3
Lisp	28	31	18	17	14	16	6	3	2
Objective-C	35	10	3	23	40	-	-	-	-
(Visual) Basic	-	-	77	5	4	4	3	5	4

为什么要学Python？



Python的特点

1. **简单、易学**
- 一种代表简单主义思想的语言，有简单的语法，容易上手。
- 伪代码本质是它最大的优点之一。
- 使你能够专注于解决问题而不是去搞明白语言本身。
2. **面向对象的高层语言**
- 无需关注底层细节，而C/C++中需要操作指针。
- 与其他语言相比，以强大而又简单的方式实现面向对象编程。
3. **解释性**
- 不需要被编译成二进制代码，可以直接在源代码上运行。
- 对于编译性语言（C/C++），源文件->编译/链接器->可执行文件。

Python的特点

4. 免费开源，可移植性

- Unix衍生系统，Win32系统家族，掌上平台（掌上电脑/手机），游戏控制台（PSP）等等。

5. 可扩展性，可嵌入性

- 如果一段关键代码希望运行得更快或者希望算法不公开，你可以把这部分程序用C或C++编写，然后在Python程序中使用。
- 可以把Python嵌入到C/C++程序中，从而向程序用户提供脚本功能。

6. 丰富的库

- Python标准库确实很庞大，包括正则表达式、文档生成、单元测试、线程、数据库、网页浏览器等等。此外还有其他高质量的库，如wxPython、Twisted和图像库等等。

如何高效学习Python?

不需要具有计算机专业的学位
不需要上一门完整的Python编程课
不需要记住所有的语法

自顶向下学习法：

- 先学习核心的编程概念
- 再学习使用一些相关的库
- 最后通过实际项目来运用知识和改进技术

Python 2还是Python 3？

Python 3不向下兼容Python 2，即Python 2中的一些函数和包不能在Python 3中使用

2010年发布的Python 2.7是Python 2最后的版本，Python 2将不再更新

2020年1月1日起，Python核心开发团队将不再对Python 2提供任何官方支持

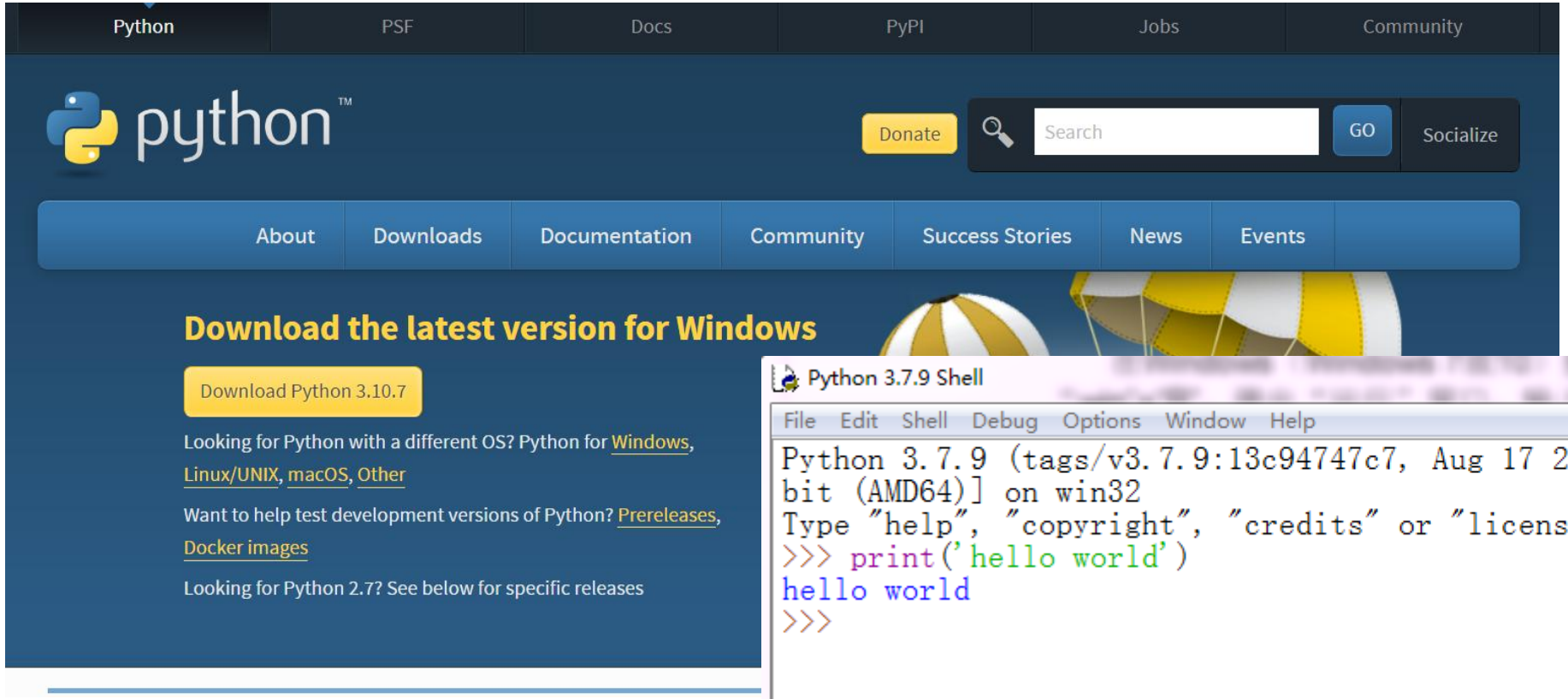
Python 3是未来的主流

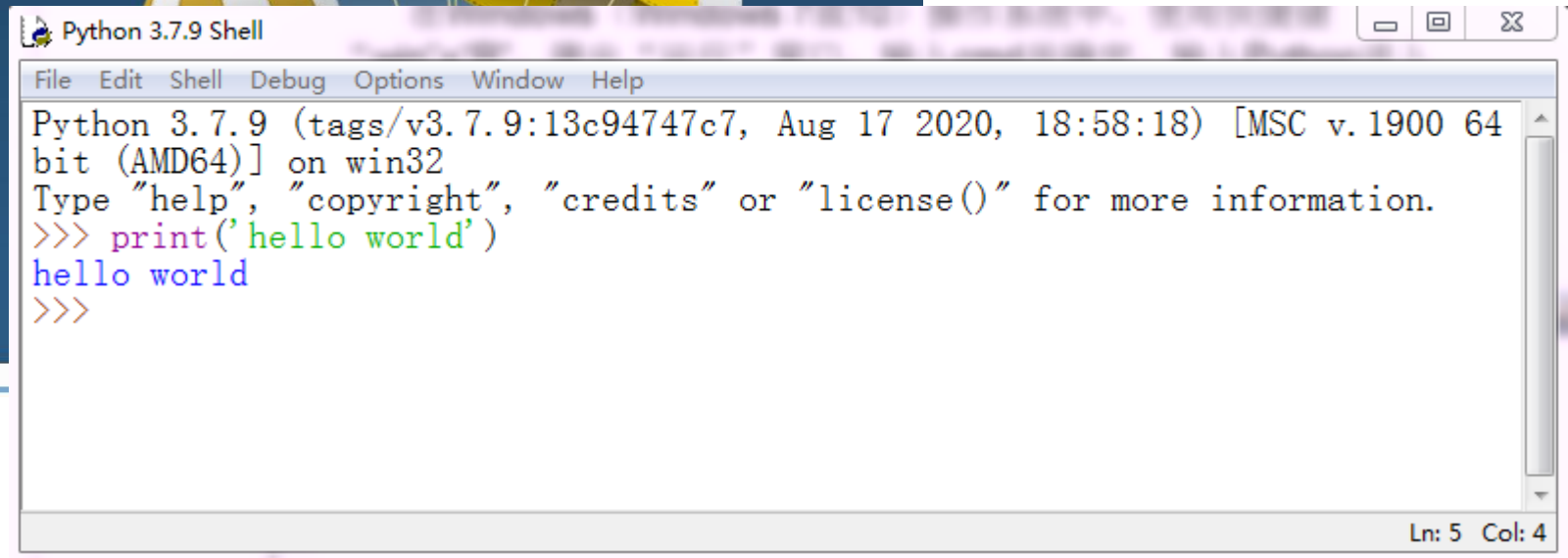
本课程中使用Python 3

1. Python简介
2. **Python开发环境**
3. Python基础编程
4. 相关库介绍

Python的开发环境

进入Python官方网站（https://www.python.org）下载安装包





Python的开发环境

Anaconda

- 一个用于科学计算的Python发行版
- 提供了包管理与环境管理的功能，可以很方便地解决多版本python并存、切换以及各种第三方包安装问题
- 支持 Linux, Mac OS, Windows系统
- 下载：https://www.anaconda.com

安装路径不要有中文等特殊字符

Miniconda Installers

Windows

Python 3.12

- 64-Bit Graphical Installer

Mac

Python 3.12

- 64-Bit (Apple silicon) Graphical Installer
- 64-Bit (Apple silicon) Command Line Installer
- 64-Bit (Intel chip) Graphical Installer
- 64-Bit (Intel chip) Command Line Installer

Linux

Python 3.12

- 64-Bit (x86) Installer
- 64-Bit (AWS Graviton2 / ARM64) Installer
- 64-bit (Linux on IBM Z & LinuxONE) Installer

Anaconda Installers

Download

Windows

Python 3.12

- 64-Bit Graphical Installer (912.3M)

Mac

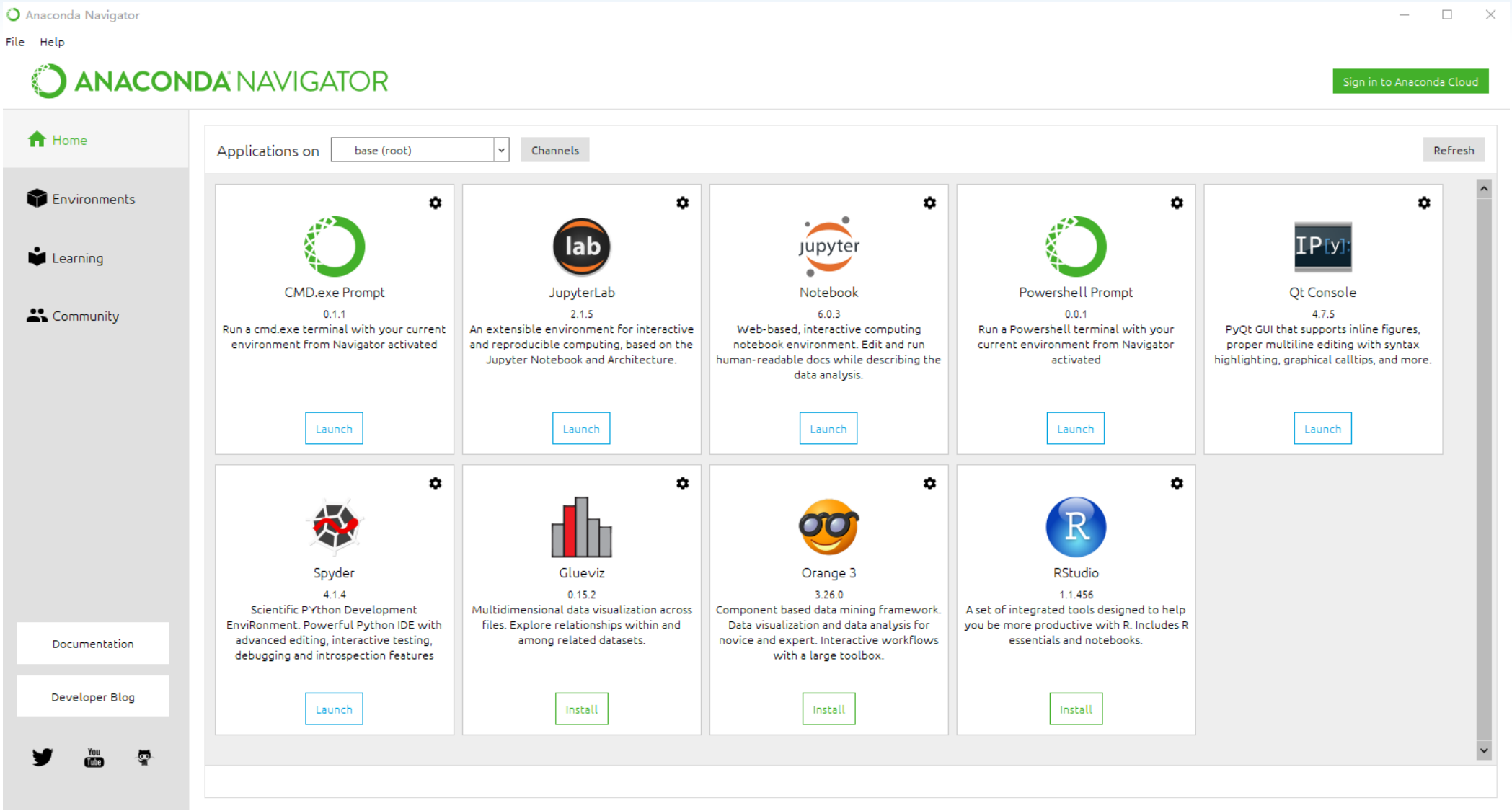
Python 3.12

- 64-Bit (Apple silicon) Graphical Installer (704.7M)
- 64-Bit (Apple silicon) Command Line Installer (707.3M)
- 64-Bit (Intel chip) Graphical Installer (734.7M)
- 64-Bit (Intel chip) Command Line Installer (731.2M)

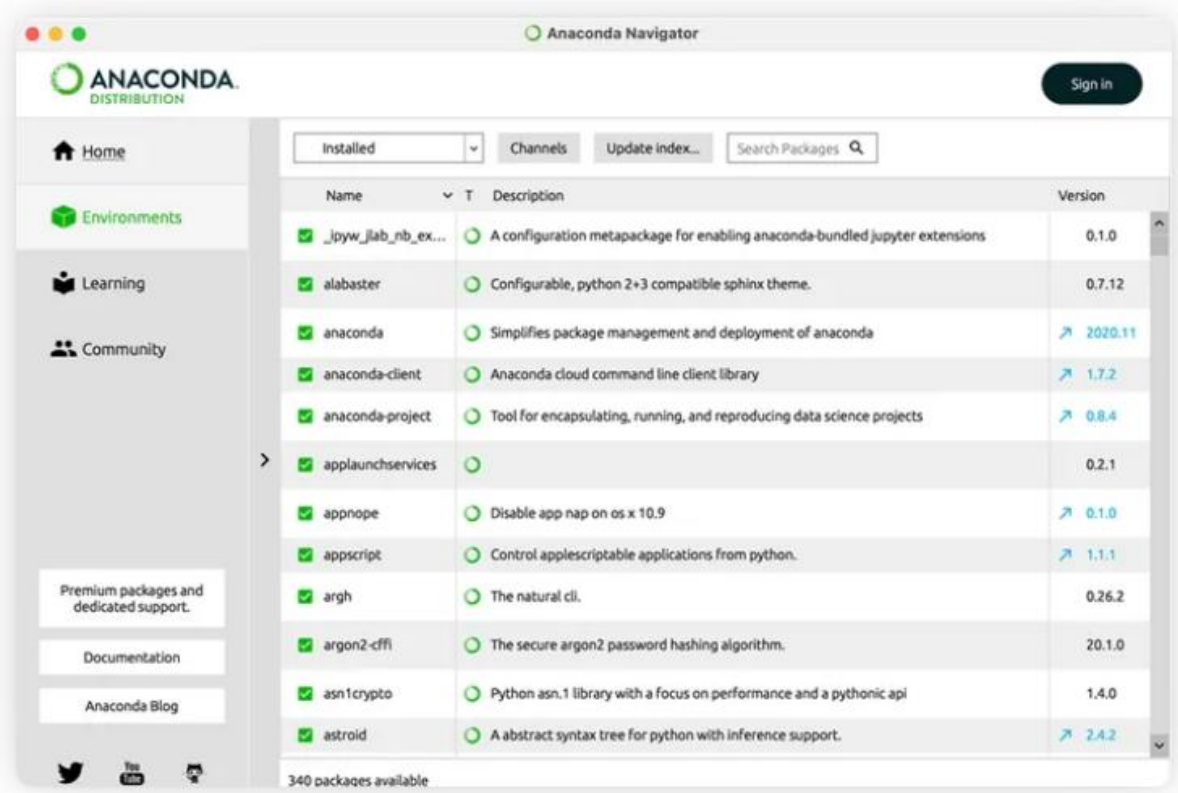
Linux

Python 3.12

- 64-Bit (x86) Installer (1007.9M)
- 64-Bit (AWS Graviton2 / ARM64) Installer (800.6M)
- 64-bit (Linux on IBM Z & LinuxONE) Installer (425.8M)


Anaconda Navigator
File Help
ANACONDA NAVIGATOR
Sign in to Anaconda Cloud
Home
Environments
Learning
Community
Documentation
Developer Blog
Applications on
base (root)
Channels
Refresh
CMD.exe Prompt
0.1.1
Run a cmd.exe terminal with your current environment from Navigator activated
Launch
JupyterLab
2.1.5
An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.
Launch
Notebook
6.0.3
Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.
Launch
Powershell Prompt
0.0.1
Run a Powershell terminal with your current environment from Navigator activated
Launch
Qt Console
4.7.5
PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.
Launch
Spyder
4.1.4
Scientific PYthon Development EnviRonment. Powerful Python IDE with advanced editing, interactive testing, debugging and introspection features
Launch
Glueviz
0.15.2
Multidimensional data visualization across files. Explore relationships within and among related datasets.
Install
Orange 3
3.26.0
Component based data mining framework. Data visualization and data analysis for novice and expert. Interactive workflows with a large toolbox.
Install
RStudio
1.1.456
A set of integrated tools designed to help you be more productive with R. Includes R essentials and notebooks.
Install

jupyter
SciPy
SPYDER
NumPy
DASK
bokeh
scikit learn
Numba
pandas
TensorFlow
Panel
matplotlib
PYTORCH
HoloViews
CONDA

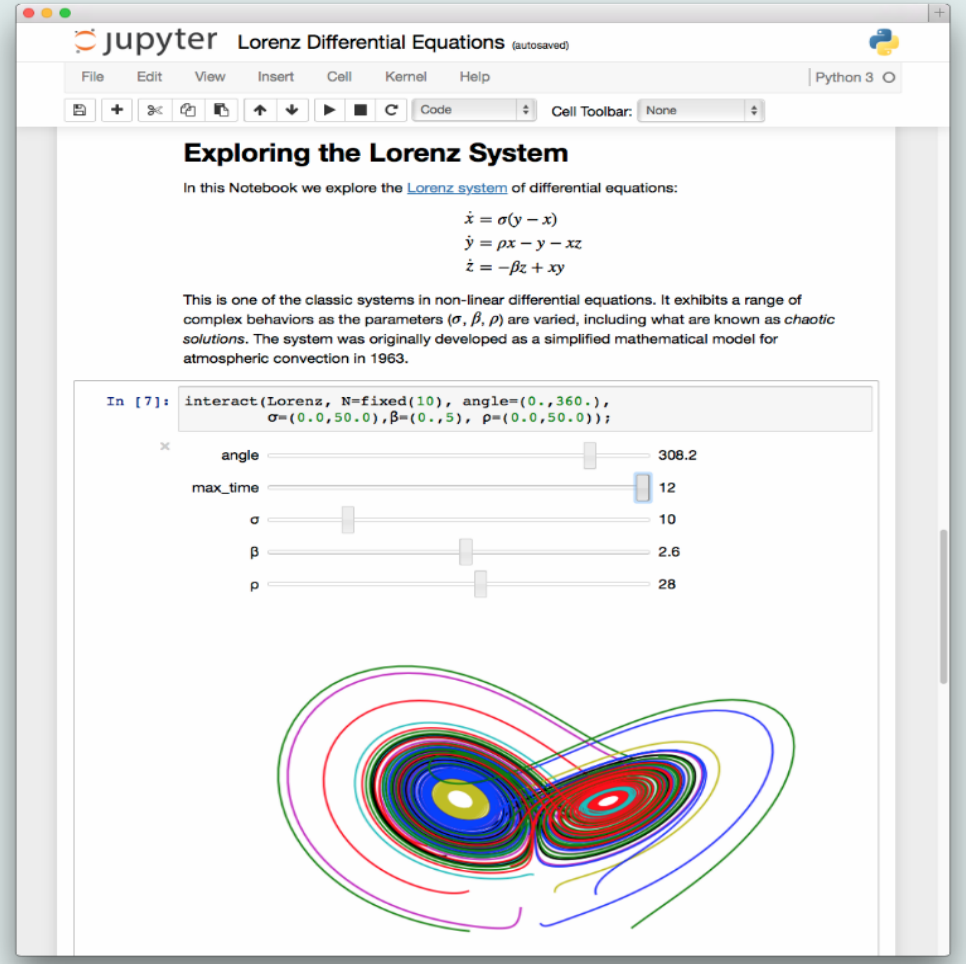

Anaconda Navigator
ANACONDA. DISTRIBUTION
Sign in
Home
Environments
Learning
Community
Installed
Channels
Update index...
Search Packages
Name
T
Description
Version
_ipyw_jlab_nb_ex...
A configuration metapackage for enabling anaconda-bundled jupyter extensions
0.1.0
alabaster
Configurable, python 2+3 compatible sphinx theme.
0.7.12
anaconda
Simplifies package management and deployment of anaconda
2020.11
anaconda-client
Anaconda cloud command line client library
1.7.2
anaconda-project
Tool for encapsulating, running, and reproducing data science projects
0.8.4
applaunchservices
0.2.1
appnope
Disable app nap on os x 10.9
0.1.0
appscript
Control applescriptable applications from python.
1.1.1
argh
The natural cli.
0.26.2
argon2-cffi
The secure argon2 password hashing algorithm.
20.1.0
asn1crypto
Python asn.1 library with a focus on performance and a pythonic api
1.4.0
astroid
A abstract syntax tree for python with inference support.
2.4.2
Premium packages and dedicated support.
Documentation
Anaconda Blog
340 packages available

Anaconda里的包

- **NumPy**（ http://www.numpy.org/ ）：用于处理（大）数组
- **Pandas**（http://pandas.pydata.org/）：数据分析工具包
- **Matplotlib**（http://matplotlib.org）：用于绘制图表
- SciPy（http://www.scipy.org）：包含许多有用的科学函数
- **Scikit-learn**（http://scikit-learn.org/）：机器学习算法
- **IPython**（http://ipython.org/）：基于Shell或浏览器的开发环境
- Spyder（ https://www.spyder-ide.org/ ）：交互式集成开发环境

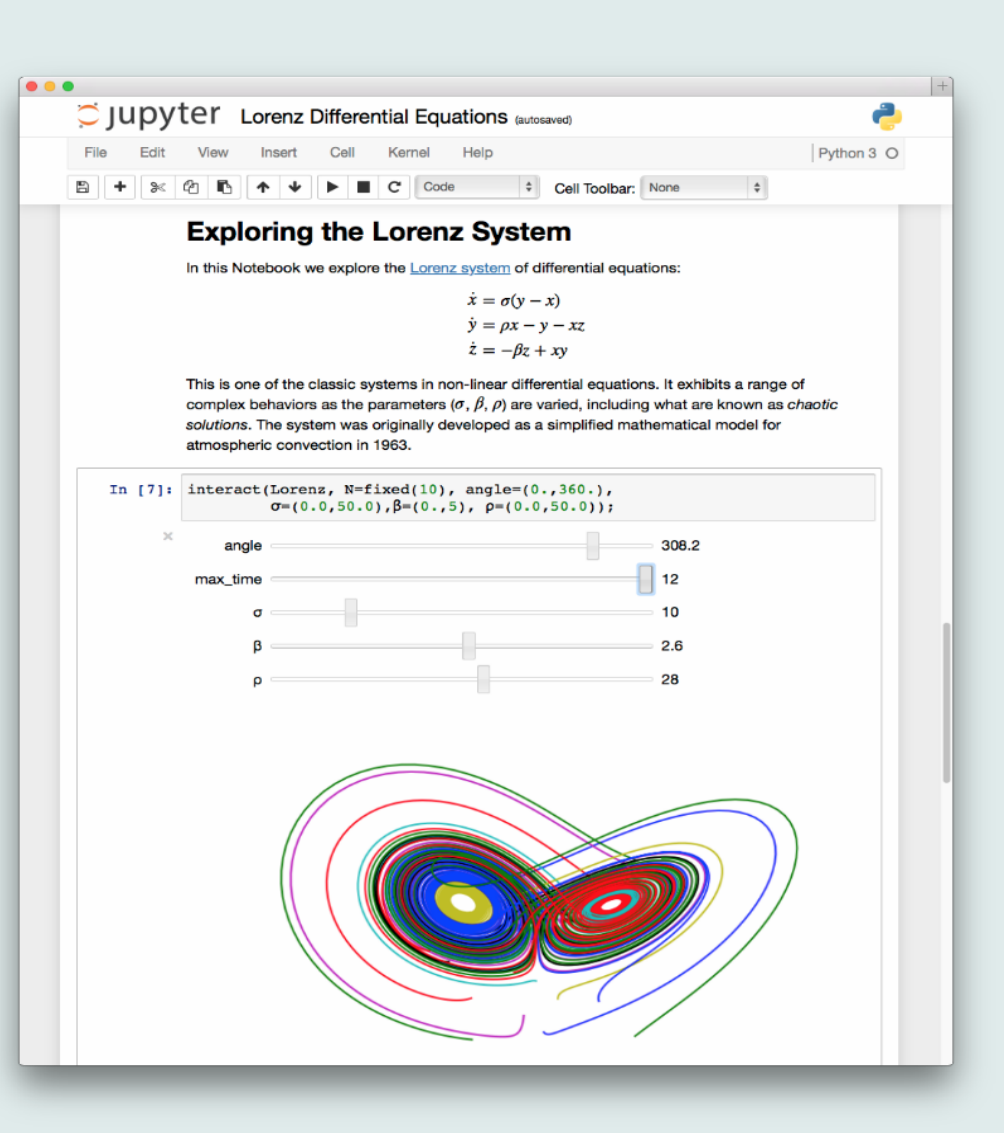
Jupyter Notebook

- 此前被称为 IPython Notebook
- 多语分析环境—支持40多种编程语言
- Jupyter 是 Julia, Python 和R几个词的变位词
- 支持多种内容类型：代码、描述文本、图像、视频等等
- http://jupyter.org/



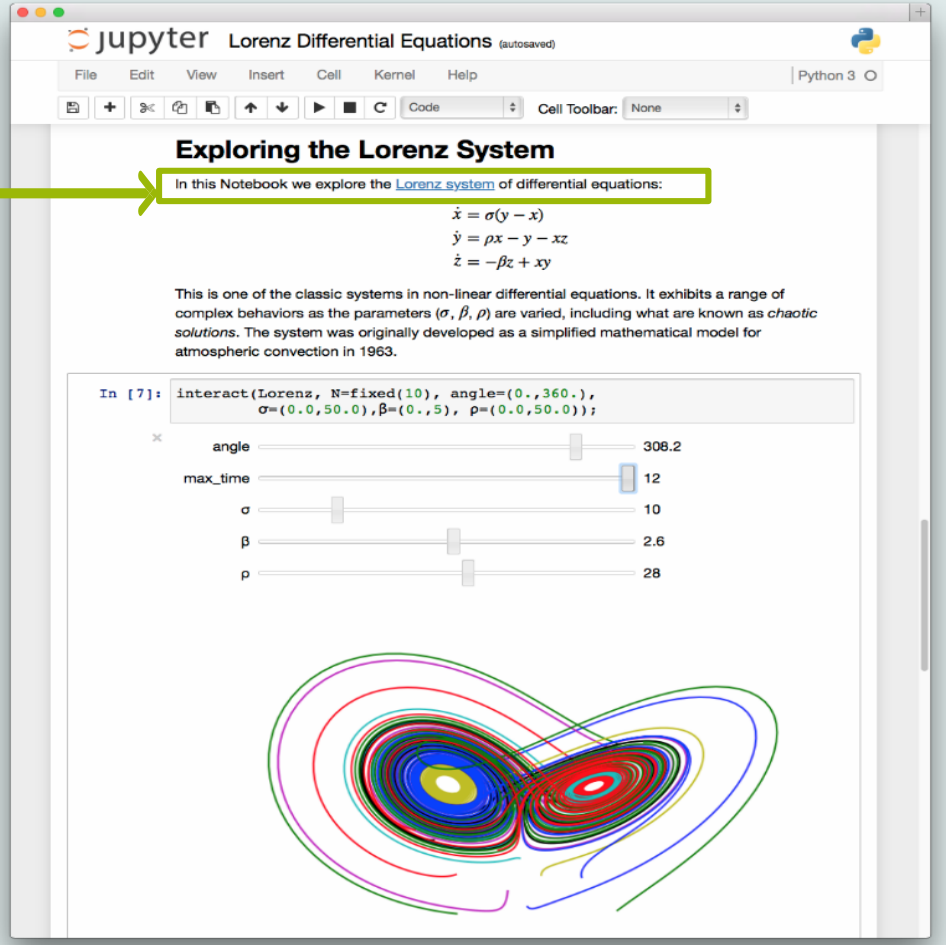
Jupyter Notebook

- HTML & Markdown
- LaTeX（公式）
- Code（代码）



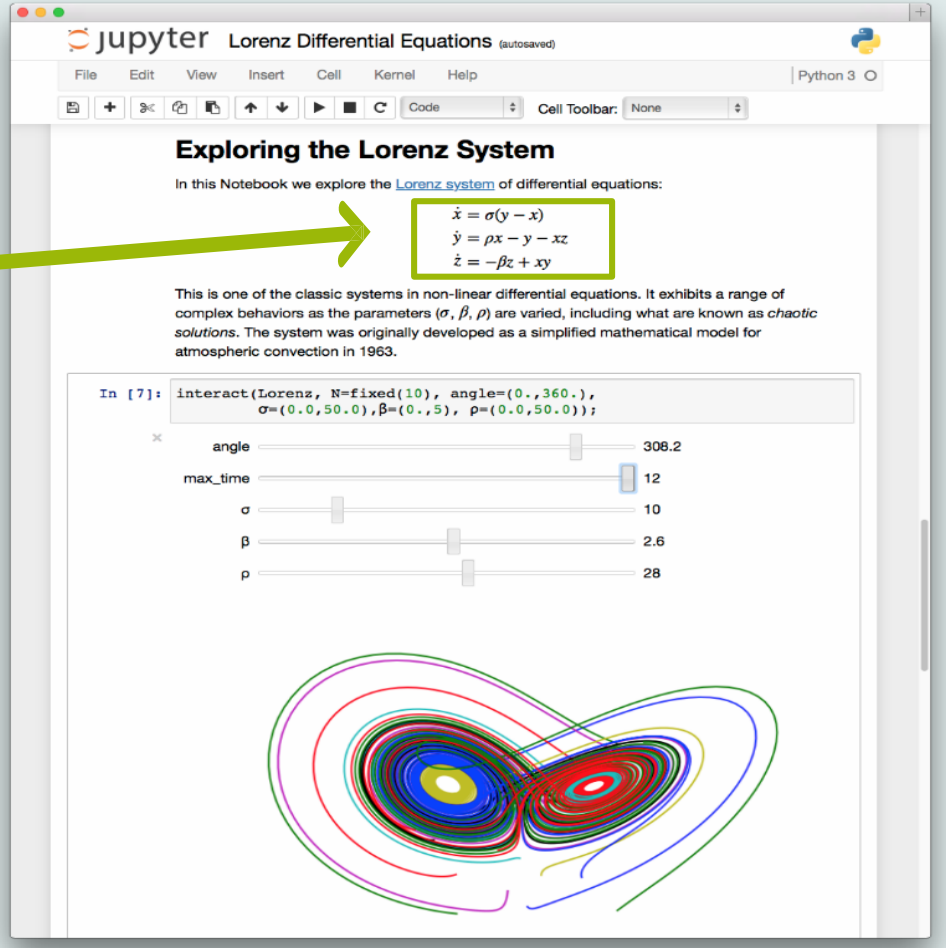
Jupyter Notebook

- HTML & Markdown
- LaTeX（公式）
- Code（代码）



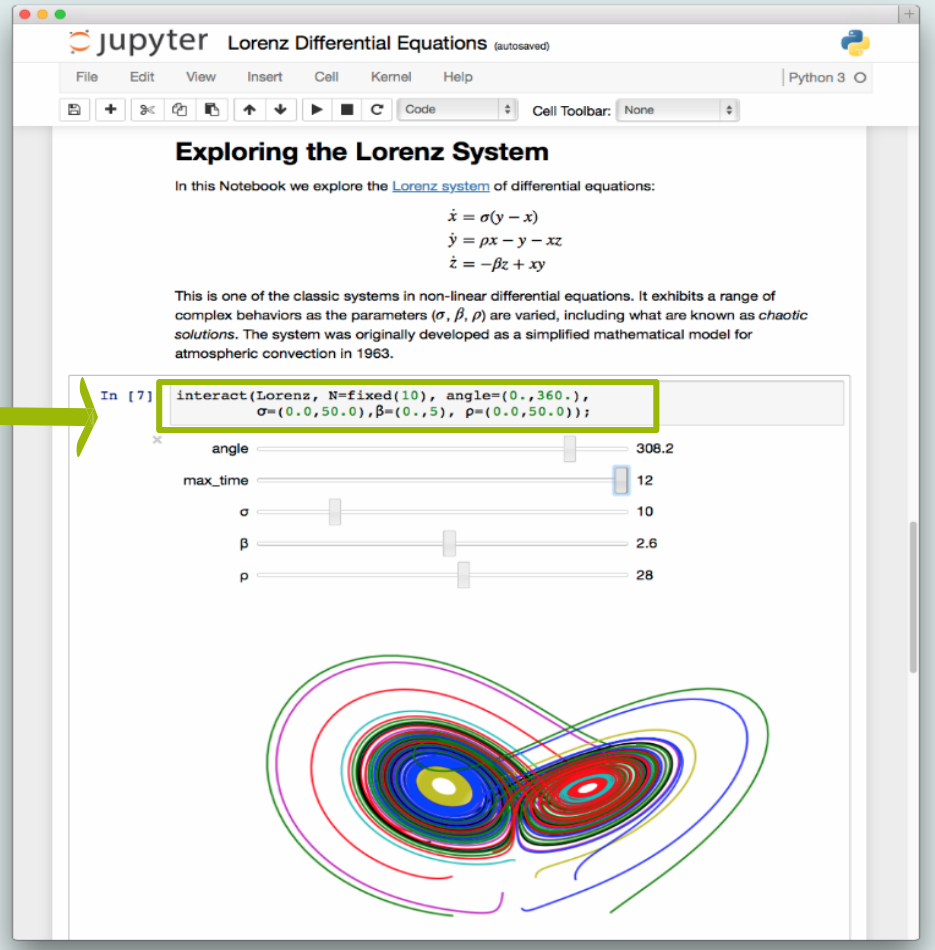
Jupyter Notebook

- HTML & Markdown
- LaTeX（公式）
- Code（代码）



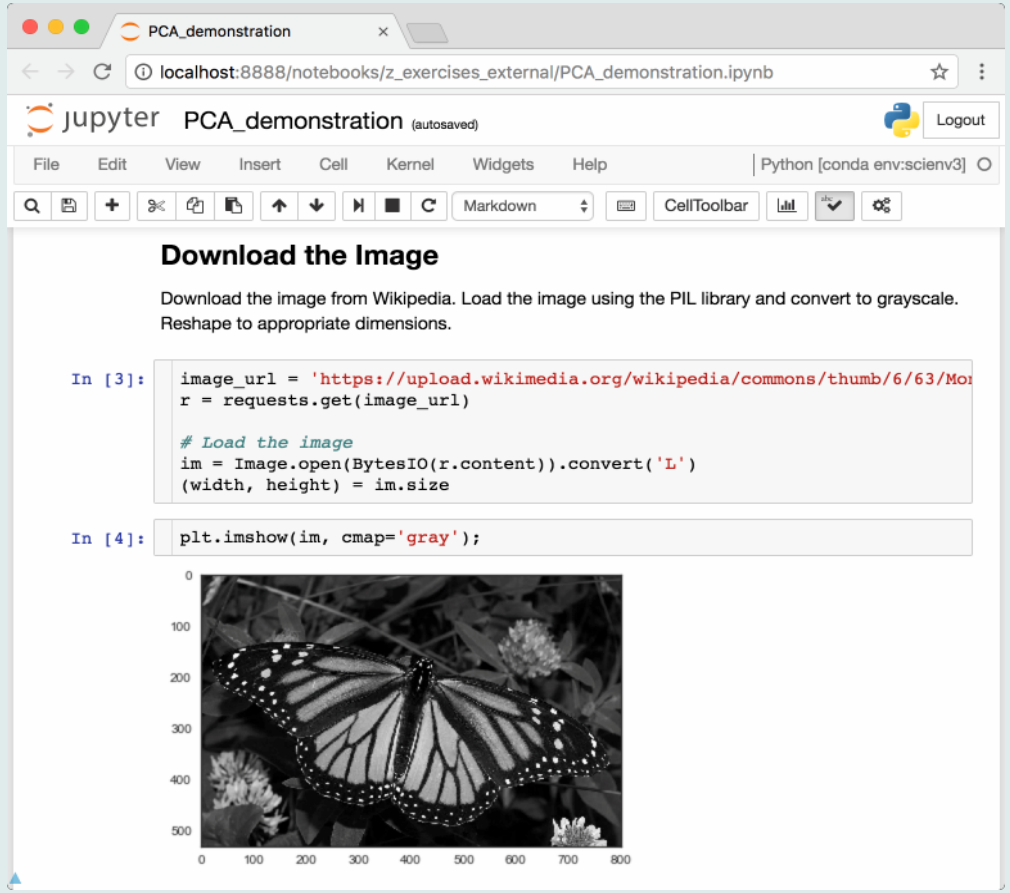
Jupyter Notebook

- HTML & Markdown
- LaTeX（公式）
- Code（代码）



Jupyter Notebook

- 代码被划分成多个单元，可以控制执行过程
- 允许进行交互式开发
- 非常适合探索式分析与建模

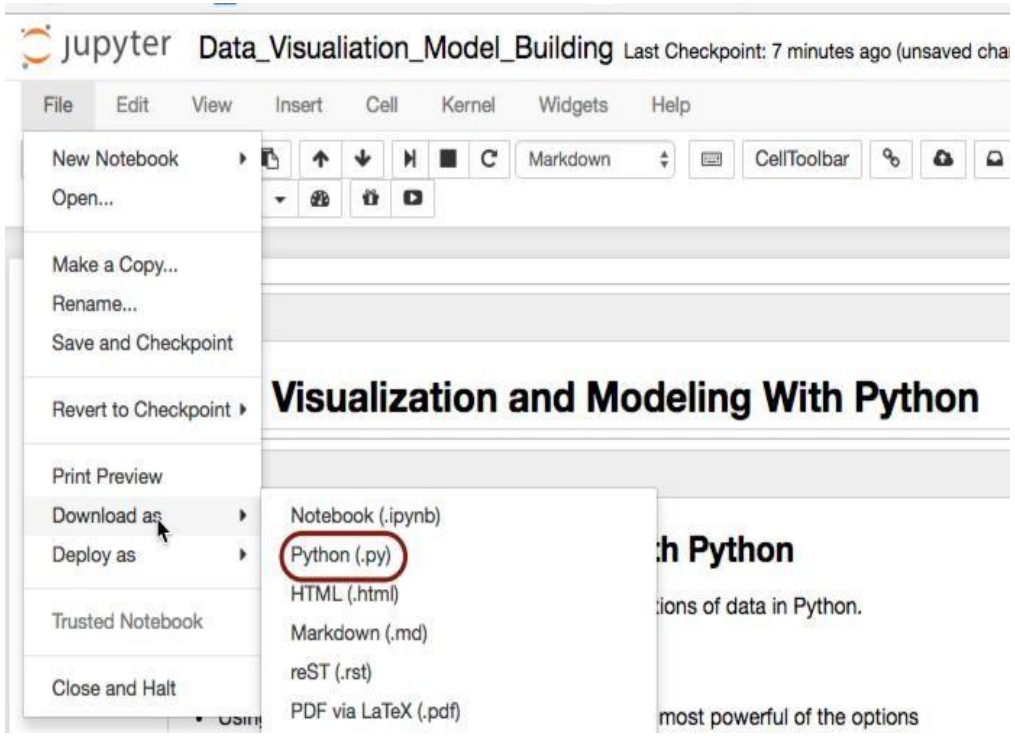


抽取Python代码

从命令行转换：

```
>>> jupyter nbconvert --to python notebook.ipynb
```

从Notebook导出代码：



Jupyter Notebook参考资料

Jupyter官网：https://jupyter.org/

Jupyter Notebook快速入门：

- http://codingpy.com/article/getting-started-with-jupyter-notebook-part-1/
- http://codingpy.com/article/getting-started-with-jupyter-notebook-part-2/

Markdown语法说明：

- https://www.appinn.com/markdown/

1. Python简介
2. Python开发环境
3. **Python基础编程**
   - 数据类型和变量
   - 运算符和表达式
   - 容器类型
   - 控制流语句
   - 函数
   - 文件操作
4. 相关库介绍

Python基本语法

一个语句占一行

- 单个语句占多行，用反斜杠\
- 多个语句在一行，使用分号;分隔

Python使用缩进表示代码块，而不是一对花括号{}

注释：

- 单行注释用#
- 多行注释用三个单引号 ''' 或者三个双引号 """ 将注释括起来

Hello World!

print函数：

```
print(“Hello World!”)
```

数据类型（1）

基本数据类型：

1. 数字（number）
   a) 整数（int）：*2，59，100，-3*
   b) 小数（float）：*0.4，5.0，-0.78*
2. 布尔（bool）：*True，False*
3. 字符串（str）：*”How are you?”，’this is a string.’*

数据类型（2）

容器类型:

1. 列表（list）：*[1, 2, 5, 10]*
2. 元组（tuple）：*(1, ‘two’)*
3. 集合（set）：*{‘Mike’, ‘John’, ‘Marry’}*
4. 字典（dictionary）：*{‘20120010’: 98, ‘20120011’: 89, ‘20120023’:100}*

变量

- 变量在被赋值的时候创建，无需声明
- 变量可以在任何时候被重新赋值为任何其他类型的值

```
message = “Hello World!”
print(message)

message = “Hello Python Course!”
print(message)
type(message)

message = 3.6
print(“Python “, message)
type(message)

message = 100
print(“There are “, message, “cars”)
type(message)
```

```
a, b, c, d = 20, 5.5, True, 4+3j
print(type(a), type(b), type(c), type(d))
```

变量命名规则

- 变量名只能包含字母、数字和下划线。
- 变量名只能以字母或下划线开头。
- 变量名是大小写敏感的。
- 变量名不要使用Python的关键字或函数名

下面哪些变量名是不合法的：
current balance, current_balance, currentBalance,
current-balance,
4account, account4, _spam, _Spam, print, ‘hello’

算术表达式

基本的算术运算符：+、-、*、/、//、%、**

```
5 + 4      # 加法

4.3 - 2    # 减法

3 * 7      # 乘法

2 / 4      # 除法，得到浮点数

2 // 4     # 除法，得到整数

17 % 3     # 取余

2 ** 5     # 乘方
```

```
x = 3
print(type(x)) # Prints "<class 'int'>"
print(x)
print(x + 1)
print(x - 1)
print(x * 2)
print(x ** 2)

x += 1
print(x)
x *= 2
print(x)

y = 2.5
print(type(y)) # Prints "<class 'float'>"
print(y, y + 1, y * 2, y ** 2)
```

布尔表达式

‘True’ 和 ‘ False’为预定义值；实际上是整数1 和 0

比较运算符：<、<=、>、>=、==、!=，结果是布尔值

布尔运算符：not、and、or

```
12 < 13

12 > 13

12 <= 12

12 != 13

True * 12

0 and 1

(3+2) < (5-7)

(7==6) and (12!=21)
```

```
t = True
f = False

print(type(t)) # Prints "<class 'bool'>"
print(t and f) # Logical AND;
print(t or f)  # Logical OR;
print(not t)   # Logical NOT;
print(t != f)  # Logical XOR;
```

字符串

字符串用单引号’或双引号”括起来

使用反斜杠\转义特殊字符：\n 换行符，\t 制表符，……

三引号用于大块的文本内容

```
a = 'Hello world!'
b = "Hello world!"
a == b

a = "Per's lecture"
print(a)
```

```
a = "One line.\nAnother line."
print(a)

b = """One line,
another line."""
print(b)
```

字符串运算

```
# 字符串拼接
a = "Part 1"
b = "and part 2"
a + ' ' + b

# 字符串重复并拼接
s = a * 2
print(s)

# 提取子串
s[0]
print(s[0:4])
print(s[5:])
print(s[6:-1])

# 字符串长度
len(s)

# 子串检测
'p' in s
'P' in s
'Part' in s
```

字符串不能改变

- Python中的字符串不可修改！

```
# 直接修改字符串会报错
s[0] = 'B'
```

- 如果要改变一个字符串：用旧的字符串片段生成一个新的

```
# 构建新字符串
s = 'B' + s[1:]
```

- 如果要生成许多新串，尝试字符串格式化

```
hello = 'hello'
world = "world"
# 字符串格式化
hw12 = '%s %s %d' % (hello, world, 12)
print(hw12)
```

- 列表（List）处理能让字符串处理更为有效

字符串的方法

- 字符串有一组内建（built-in）方法
- 没有方法可以改变原串，有几个方法可以生成新串

```
s = 'a string, with stuff'
s.count('st')                      # 有多少子串？
s.find('stu')                      # 寻找子串，如果有，给出子串的位置
s.replace('stuff', 'characters')   # 替换子串 (全部出现过的子串)
s.replace('s', 'X', 1)             # 只替换一次

s = '3'
s.isdigit()                        # 是纯数字串吗？
```

```
s = "hello"
print(s.capitalize())          # 首字母大写；输出"Hello"
print(s.upper())               # 所有字符转换成大写字符；输出"HELLO"
print(s.rjust(7))              # 右对齐，左端补空格；输出"  hello"
print(s.center(7))             # 居中对齐，左右两端补空格；输出" hello "

print('  world ').strip())     # 去除前后的所有空白符；输出"world"
```

列表

- 有序的对象序列
- 异质的；可以包含任意类型的对象的混合

```
r = [1, 2.0, 3, 5]          # 列表实例，不同的值
type(r)                     # 输出<class 'list'>
```

```
r[1]                        # 通过下标来访问；偏移量为 0
r[-1]                       # 负的下标代表从尾部开始计数
r[1:3]                      # 列表的片段；给出新的列表
```

```
w = r + [10, 19]            # 合并列表；给出另外的一个列表
w
r                           # 原列表不变；w 和 r 不同
```

```
t = [0.0] * 10              # 用重复生成一个初始向量
t
```

列表操作

- 列表是可变的，可以改变局部

```
r = [1, 2.0, 3, 5]
r[3] = 'word'                # 通过下标改变一个元素（项）
r                            # 显示[1, 2.0, 3, 'word']
```

```
r[0] = [9, 8]                # 列表可以嵌套
r                            # 显示[[9, 8], 2.0, 3, 'word']
```

```
r[0:3] = [1, 2, 5, 6]        # 改变列表的一个片段，可以改变列表的长度
r                            # 显示[1, 2, 5, 6, 'word']

r[1:3] = []                  # 通过设置列表的片段为空集来移除元素
r                            # 显示[1, 6, 'word']
```

```
len(r)                       # 列表的长度，即项的个数，显示3
```

```
6 in r                       # 成员测试，显示True
r.index(6)                   # 搜索并给出位置，如果没有的话，报错，这里显示1
```

列表的方法（1）

```
r = [1, 2.0, 3, 5]
r.append('thing')                    # 在列表尾增加一个项
r                    # 显示[1, 2.0, 3, 5, 'thing']

r.append(['another', 'list'])        # 增加的列表被看作一个单一项
r                    # 显示[1, 2.0, 3, 5, 'thing', ['another', 'list']]
```

```
r = [1, 2.0, 3, 5]
r.extend(['item', 'another'])  # 列表的项逐次添加
r                    # 显示[1, 2.0, 3, 5, 'item', 'another']
```

```
k = r.pop()                          # 移除最后一项
k                                    # 显示'another'
r                                    # 显示[1, 2.0, 3, 5, 'item']

r.insert(3, 4.0)                     # 在指定位置插入一项
r                                    # 显示[1, 2.0, 3, 4.0, 5, 'item']

r.remove('item')                     # 删除一项
r                                    # 显示[1, 2.0, 3, 4.0, 5]
```

列表的方法（2）

- 使用内建的sort方法：排序是内部进行的，不产生新列表！
- 外部函数sorted，不改变原列表的顺序

```
r = [2, 5, -1, 0, 20]
r.sort()
r                                # 显示[-1, 0, 2, 5, 20]

w = ['apa', '1', '2', '1234']
w.sort()                         # 字符串：使用ASCII顺序
w                                # 显示['1', '1234', '2', 'apa']
```

```
w.reverse()                      # 反转列表！
w                                # 显示['apa', '2', '1234', '1']

v = w[:]                         # 首先生成新表
v.reverse()                      # 反转这份拷贝
v                                # 显示['1', '1234', '2', 'apa']
w                                # 显示['apa', '2', '1234', '1']
```

转换字符串为列表

```
s = 'biovitrum'            # 生成字符串
w = list(s)                # 转为字符的列表
w                    # 显示['b', 'i', 'o', 'v', 'i', 't', 'r', 'u', 'm’]
w.reverse()
w                    # 显示['m', 'u', 'r', 't', 'i', 'v', 'o', 'i', 'b’]

r = ''.join(w)             # 使用空串的join方法
r                          # 显示'murtivoib’

d = '-'.join(w)            # 使用字符-的join方法
d                          # 显示'm-u-r-t-i-v-o-i-b‘

s = 'a few words'
w = s.split()              # 基于空白符(空格, 新行)切分
w                          # 显示['a', 'few', 'words’]

‘ | ’.join(w)              # 对其他串用方法‘join‘, 显示'a | few | words'
```

元组

- 和列表一样，除了不可变，即一旦生成，就不可改变
- 某些函数会返回元组

```
t = (1, 3, 2)
t[1]                          # 由下标访问，偏移量从0开始，显示3

(a, b, c) = t                 # 元组赋值
a                             # 显示1
b                             # 显示3

a, b, c                       # 一个实际上的元组表达式!显示(1, 3, 2)
```

```
a, b = b, a                   # 交换值的技巧
a, b                          # 显示(3, 1)
```

```
r = list(t)                   # 转换元组为列表
r                             # 显示[1, 3, 2]

tuple(r)                      # 转换列表为元组，显示(1, 3, 2)
```

集合

- 一个**无序**的**没有重复元素**的序列
- 基本功能是进行成员关系测试和删除重复元素，可以进行集合运算

```
student = {'Tom', 'Jim', 'Mary', 'Tom', 'Jack', 'Rose'}
print(student)                          # 输出集合，重复元素被自动去掉
{'Mary', 'Jim', 'Rose', 'Jack', 'Tom'}
print('Rose' in student)                # 成员测试
True

a = set('abracadabra')                  # 字符串转换成集合
b = set('alacazam')
print(a)
{'b', 'a', 'c', 'r', 'd'}
print(a - b)                            # a和b的差集
{'b', 'd', 'r'}
print(a | b)                            # a和b的并集
{'l', 'r', 'a', 'c', 'z', 'm', 'b', 'd'}
print(a & b)                            # a和b的交集
{'a', 'c'}
print(a ^ b)                            # a和b中不同时存在的元素
{'l', 'r', 'z', 'm', 'b', 'd'}
```

字典

- **键（key）值（value）**对的无序集合
- 键必须使用不可变类型。可以用数字、字符串或元组，不能用列表
- 在同一个字典中，键必须是唯一的，值不必

```
h = {'key': 12, 'nyckel': 'word'}
h[‘key’]                    # 由键访问，显示12

h.has_key(‘nyckel’)         # 显示True
```

- 增加一个键/值

```
h['Per'] = 'Kraulis'
h
{'nyckel': 'word', 'Per': 'Kraulis', 'key': 12} # 输出顺序是随机的
```

- 替换一个键对应的值

```
h[‘Per’] = ‘Johansson’
h
{'nyckel': 'word', 'Per': 'Johansson', 'key': 12}
```

字典的方法（1）

```
h = {'key': 12, 'nyckel': 'word'}
'Per' in h                    # 测试一个键是否在字典中，显示False

h['Per']                      # 报错
```

```
h.get('Per', 'unknown')       # 返回值，或者返回缺省值，显示'unknown'

h.get('key', 'unknown')       # 显示12
```

```
h.keys()                      # 字典中所有的键
['nyckel', 'key']
h.values()                    # 字典中所有的值
['word', 12]
h.items()                     # 字典中所有的键值对
dict_items([('key', 12), ('nyckel', 'word')])
```

```
len(h)                        # 字典中键值对的个数，显示2
```

删除数据的命令：del

- 命令！不是函数！
- 实际上移除变量（名字），不是对象

```
a = 'thing'                    # 定义一个变量
a                              # 显示'thing'

del a                          # 把这个变量忘掉
a                              # 报错
```

```
h = {'key': 12, 'nyckel': 'word'}
del h['key']                   # 移除键和它的值
h                              # 显示{'nyckel': 'word'}
```

```
r = [1, 3, 2]
del r[1]                       # 另一个删除列表项的方式
r                              # [1, 2]
```

字典方法（2）

```
g = h.copy()                # 拷贝字典

del h['key’]
h                           # 显示{'nyckel': 'word’}
g                           # 显示{'nyckel': 'word', 'key': 12}

h['Per'] = 'Johansson’
h                           # 显示{'nyckel': 'word', 'Per': 'Johansson’}

h.update(g)     # 根据g添加或者更新所有的键值
h               # 显示{'nyckel': 'word', 'key': 12, 'Per': 'Johansson’}

g.clear()       # 清除字典中的所有项
print(len(g))   # 显示0
```

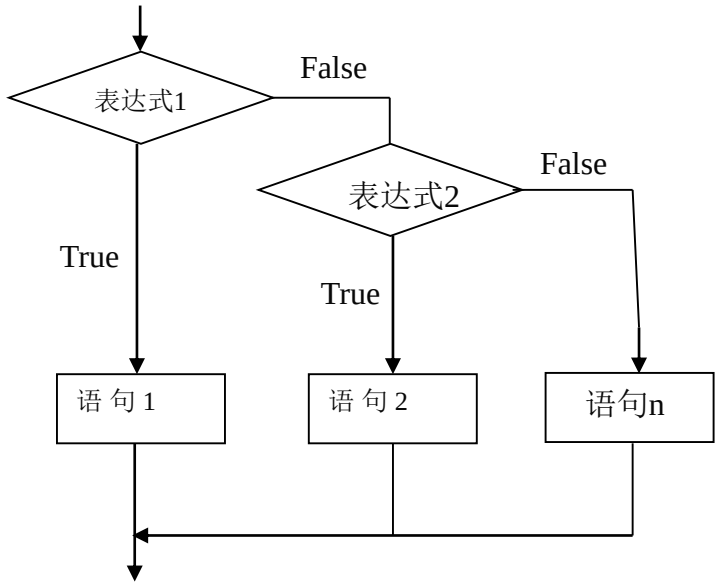
程序设计的三种基本结构：顺序结构、选择结构、循环结构

- 顺序结构：是一种线性、有序的结构，依次执行各语句模块
- 选择结构：根据条件成立与否选择程序执行的路径
- 循环结构：重复执行一个或几个模块，直至满足某一条件

条件语句：if

if语句是由if发起的一个条件语句，在满足此条件后执行相应内容，Python的语句基本结构如下。

```
if 表达式1:
    语句块1
elif 表达式2:
    语句块2
……
else:
    语句块n
```



分支选择结构

条件语句：if

```
age = 17
if age >= 18:
    print("You are old enough to vote!")
    print("Have you registered to vote yet?")
else:
    print("Sorry, you are too young to vote.")
    print("Please register to vote as soon as you turn 18!")
```

```
age = 12
if age < 4:
    print("Your admission cost is $0.")
elif age < 18:
    print("Your admission cost is $5.")
else:
    print("Your admission cost is $10.")
```

循环语句：while

while condition:
    statement_block

```
current_number = 1
while current_number <= 5:
    print(current_number)
    current_number += 1
```

```
pets = ['dog', 'cat', 'dog', 'goldfish', 'cat', 'rabbit', 'cat']
print(pets)

while 'cat' in pets:
    pets.remove('cat')
print(pets)
```

循环语句：for

for variable **in** sequence:
    statement_block
else:
    statement_block

```
magicians = ['alice', 'david', 'carolina']
for magician in magicians:
    print(magician.title() + ", that was a great trick!")
```

```
squares = []
for value in range(1,11):
    squares.append(value**2)
print(squares)
```

```
r = []
for c in 'this is a string with blanks': # 一个字符一个字符地遍历字符串
    if c == ' ': continue                # 跳过后面的代码块,继续循环
    r.append(c)
print (''.join(r))
```

循环中的break，continue和else

- break语句跳出循环
- continue语句结束本轮循环，开始下一轮循环
- else在循环条件不满足时被执行，被break的循环不执行else
- pass语句是空语句，什么都不做，占位语句

```
r = [1, 3, 10, 98, -2, 48]
for i in r:
    if i < 0:
        print ('input contains negative value!')
        break                                   # 跳出整个循环，包括'else'
    else:
        pass                                    # 什么都不做
else:                                           # 如果循环是正常结束的，则执行
    print ('input is OK')
```

列表循环与List Comprehension

```
nums = [0, 1, 2, 3, 4]
squares = []
for x in nums:
    squares.append(x ** 2)
print(squares)                          # Prints [0, 1, 4, 9, 16]
```

```
nums = [0, 1, 2, 3, 4]
squares = [x ** 2 for x in nums]
print(squares)                          # Prints [0, 1, 4, 9, 16]
```

```
nums = [0, 1, 2, 3, 4]
even_squares = [x ** 2 for x in nums if x % 2 == 0]
print(even_squares)                     # Prints [0, 4, 16]
```

```
animals = ['cat', 'dog', 'monkey']
for idx, animal in enumerate(animals):
    print('#%d: %s' % (idx + 1, animal))
```

字典循环与Dictionary Comprehension

```
d = {'person': 2, 'cat': 4, 'spider': 8}
for animal in d:
    legs = d[animal]
    print('A %s has %d legs' % (animal, legs))
```

```
d = {'person': 2, 'cat': 4, 'spider': 8}
for animal, legs in d.items():
    print('A %s has %d legs' % (animal, legs))
```

```
nums = [0, 1, 2, 3, 4]
even_num_to_square = {x: x ** 2 for x in nums if x % 2 == 0}
print(even_num_to_square)             # Prints {0: 0, 2: 4, 4: 16}
```

集合循环与Set Comprehension

```
animals = {'cat', 'dog', 'fish'}
for idx, animal in enumerate(animals):
    print('#%d: %s' % (idx + 1, animal))
```

```
from math import sqrt

nums = {int(sqrt(x)) for x in range(30)}
print(nums)                                # Prints {0, 1, 2, 3, 4, 5}
```

函数

def 函数名（参数列表）：
    函数体

```
def greet_user():
    print("Hello!")

greet_user()
```

```
def greet_user(username):
    print("Hello, " + username.title() + "!")

greet_user('jesse')
```

```
def describe_pet(animal_type, pet_name):
    print("\nI have a " + animal_type + ".")
    print("My ‟ +animal_type+ "'s name is " + pet_name.title() + ".")

describe_pet('hamster', 'harry’)
describe_pet('dog', 'willie')
```

函数参数的默认值

- 参数可以有默认值
- 当调用时没有给定参数，会采用默认值
- 有默认值的参数必须放在参数列表的最后
- 显式调用参数，可以改变参数顺序

```
def describe_pet(pet_name, animal_type='dog'):
    print("\nI have a " + animal_type + ".")
    print("My ‘ +animal_type+ "'s name is " +pet_name.title()+ ".")

describe_pet('willie’)
describe_pet(pet_name='harry', animal_type='hamster’)
describe_pet(animal_type='hamster', pet_name='harry')
```

函数返回值

- 一个函数不一定要有return语句
- 实际上，函数默认总会返回一个值：’None’
- ’None’ 是一个特殊的值，意味着 ’什么都没有’

```
def get_formatted_name(first_name, last_name):
    full_name = first_name + ' ' + last_name
    return full_name.title()

musician = get_formatted_name('jimi', 'hendrix')
print(musician)
```

```
def getMaxMin(a):
        max=a[0]
        min=a[0]
        for i in range(0,len(a)):
            if max<a[i]:
                max=a[i]
            if min>a[i]:
                min=a[i]
        return(max,min)
a_list=[5,8,0,-2,-10,4,1,99]
x,y= getMaxMin(a_list)
print(“最大值为”,x, “最小值为”,y)
```

程序运行结果如下：

最大值为99，最小值为-10

全局变量与局部变量

Python语言中的变量是有一定的工作空间的，我们把它叫做变量的作用域。变量作用域是指变量可用的范围，python与大多数程序语言一样有局部变量和全局变量之分。当变量超出作用范围时自动消亡。

在函数外面定义的变量称为全局变量。全局变量的作用域在整个代码段（文件、模块），在整个程序代码中都能被访问到。在函数内部可以去访问全局变量。

在函数内部定义的参数和变量称为局部变量，超出了这个函数的作用域局部变量是无效的，它的作用域仅在函数内部。

内置函数是解释器的一部分，它随着解释器的启动而生效；**库函数**是解释器的外部扩展，导入模块以后才能生效。一般来说，内置函数的执行效率要高于库函数。

不要使用内置函数的名字作为标识符使用（例如变量名、函数名、类名、模板名、对象名等），虽然这样做 Python 解释器不会报错，但这会导致同名的内置函数被覆盖，从而无法使用。

内置函数

abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slicea()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

类

在Python中使用class语句创建类

class语句后跟类名，缩进语句块表示类体

在Python中习惯在定义类时，将首字符大写，用于区分类名和函数名、变量名

类就是对象的属性和方法的封装，静态的特征称为属性，动态的动作称为方法。

类的定义

类通常的语法格式如下：

```
class ClassName:
    #属性
    [属性定义体]
    #方法
    [方法定义体]
```

```
class Person: # Python的类名约定以大写字母开头
    # 属性
    skincolor = "yellow"
    high = 168
    weight = 65
    #方法
    def goroad(self):
        print("人走路动作的测试……")
    def sleep(self):
        print("睡觉，晚安！")
```

类的使用

类定义好之后，就可将类实例化为对象。类实例化对象的语法格式如下：

对象名 = 类名（）

实例化对象的操作符是：等号“=”，在类实例化对象的时候，类名后面要添加一个括号“()”。

类实例化示例：

```
>>> p = Person()
```

上例将类Person实例化为对象p。

类实例化对象之后，对象就可以调用类的属性和方法，语法格式如下：

对象名 . 属性名

对象名 . 方法名

对象调用类的属性或方法的操作符是点“.”。

对象调用属性或方法的示例：

```
>>> p.goroad()
人走路动作的测试……
>>> p.skincolor
'yellow'
>>> p.sleep()
睡觉，晚安！
>>> p.high
168
>>> p.weight
65
```

模块

- 将函数存储在被称为模块的独立文件中，再将模块导入到主程序中
- 数学函数在一个单独的模块中

```
import math                          # 导入整个'math'模块
print (math.e, math.pi)
print (math.cos(math.radians(180.0)))
print (math.log(10.0))
print (math.exp(-1.0))
```

```
from math import *                   # 导入模块'math'中的所有函数
print (e, pi)
print (cos(radians(180.0)))
print (log(10.0))
print (exp(-1.0))
```

```
from math import log, cos            # 导入'math'模块中的log和cos函数
```

```
from math import log as lg           # 导入'math'模块中的log函数，起别名lg
```

在python中，模块可以将程序代码和数据封装起来以便重用，模块比函数粒度更大，一个模块可以包含若干个函数。与函数相似，模块也分为系统模块和用户自定义模块。一个用户自定义的模块就是一个.py文件。

我们写的代码保存的以.py结尾的Python文件就是一个独立的模块，模块包含了对象定义和语句。

模块导入方法

要导入系统模块或者已经定义好的模块，有三种方法：

1、最常用的方法是：

import module

module——是模块名，如果有多个模块，模块名称之间用逗号“,”隔开。

导入模块后，就可以引用模块内的函数，语法格式如下：

模块名.函数名

2、第二种方法是：

from 模块名 import 函数名

函数名如果有多个，可用逗号“，”隔开。

函数名可用通配符“*”导出所有的函数。

这种方法要慎用，因为导出的函数名称容易和其它函数名称冲突，失去了模块命名空间的优势。

3、第三种方法是：

import 模块名 as 新名字

这种导入模块的方法，相当于给导入的模块名称重新起一个别名，便于记忆，很方便地在程序中调用。

自定义模块和包

1. 自定义模块：

自定义模块的方法和步骤如下：

新建一个以.py为后缀名的文件，然后编辑该文件。

在自定义模块时，有几点要注意：

(1)为了能找到我们自定义模块，该模块要和调用的程序在同一目录下，否则在导入模块时会提示找不到模块的错误。

(2)模块名要遵循Python变量命名规范，不要使用中文、特殊字符等。

(3)自定义的模块名不要和系统内置的模块名相同。

2. 自定义包：

在大型项目开发中，有多个程序员协作共同开发一个项目，为了避免模块名重名，Python引入了按目录来组织模块的方法，称为包（Package）。包是一个分层级的文件目录结构，它定义了由模块及子包，以及子包下的子包等组成的命名空间。

在自定义包时，需要注意：

(1)每个包目录下面都会有一个__init__.py的文件，这个文件是必须存在的，否则，系统就把这个目录作为普通目录，而不是一个包。

(2)__init__.py可以是空文件，也可以有Python代码。

(3)在Python中可以有多级目录，组成多层次的包结构。

文件操作：读

- 一个文件操作对象由内建函数 ’open’ 创建
- 文件对象有一系列函数
- ‘read’：读取整个文件 （或者说N 字节），返回一个单独的字符串
- ‘readline’：读取一行（然后跳到新的一行）
- ‘readlines’：读取所有的行，返回一个字符串的列表

```
f = open('test.txt')    # 默认: 只读模式

line = f.readline()     # 读一行
line                    # 显示'This is the first line.\n’

lines = f.readlines()   # 读所有剩余行
lines                   # 显示['This is the second.\n', 'And third.\n']
```

文件操作：写

- ‘write’ 函数只是简单地输出给定字符串
- 字符串不一定是ASCII码，二进制串也可以

```
w = open('output.txt', 'w')          # 写模式 (默认写的是文本)

w.write('stuff')                     # 并不自动添加新行
w.write('\n')
w.write('more\n and even more\n’)

w.close()
```

```
stuff
more
 and even more
```

用for循环读取文件

```
infile = open('test.txt')                    # 只读模式
outfile = open('test_upper.txt', 'w')        # 写模式；创建文件

for line in infile:                          # 遍历文件中的每一行
    outfile.write(line.upper())

infile.close()                               # 并不严格要求；系统会自动执行
outfile.close()
```

- 注意：每行结尾会尾随一个换行符 ‘\n’
- 可以使用字符串方法’strip’或者’rstrip’去除它

参考资料

Python文档：https://docs.python.org/3/

https://www.runoob.com/python/python-lists.html

1. Python简介
2. Python开发环境
3. Python基础编程
4. **相关库介绍**

几个相关的库

Numpy：高效地处理高维数组；高效的数学函数

- Quickstart tutorial：https://docs.scipy.org/doc/numpy/user/quickstart.html
- A Visual Intro to NumPy and Data Representation：http://jalammar.github.io/visual-numpy/

Matplotlib：可视化，绘制2D或3D图形

- Pyplot tutorial：https://matplotlib.org/users/pyplot_tutorial.html

Pandas：统计与数学分析

- 10 Minutes to pandas：http://pandas.pydata.org/pandas-docs/stable/getting_started/10min.html

Scikit-learn：机器学习

- https://scikit-learn.org/stable/documentation.html

Numpy

Numpy（Numerical Python extensions）是一个第三方的Python包，用于科学计算，前身是1995年就开始开发的一个用于数组运算的库

极大地简化了向量和矩阵的操作处理，是一些主力软件包（如scikit-learn、Scipy、pandas和tensorflow）架构的基础部分。

ndarray数据类型

Numpy提供了一种新的数据结构：**ndarray**（n维数组，n-dimensional array）

不同于列表和元组，数组只能存放**相同类型**的对象（如全部整型或全部浮点型）

这使得在数组上的一些运算远远快于在列表上的相同运算；另外，数组占用更小的存储

数组强有力地扩展了列表的索引机制

创建ndarray

首先导入Numpy库

```
>>>import numpy as np
```

然后开始创建n维数组

```
>>>np.array([2, 3, 6, 7])
array([2, 3, 6, 7])

>>>np.array([2, 3, 6, 7.])
array([2., 3., 6., 7.])

>>>np.array([2, 3, 6, 7+1j])
array([2.+0.j, 3.+0.j, 6.+0.j, 7.+1.j])
```

创建等差数列的数组

arange([start,] stop[, step,], dtype=None)

```
>>>np.arange(5)
array([0, 1, 2, 3, 4])

>>>np.arange(10, 100, 20, dtype=float)
array([10., 30., 50., 70., 90.])
```

linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None)

```
>>>np.linspace(0., 2.5, 5)
array([0., 0.625, 1.25, 1.875, 2.5])

>>> from numpy import pi
>>> x = np.linspace( 0, 2*pi, 100 )  # 用于在多个点执行某函数
>>> f = np.sin(x)
```

多维数组表示的矩阵

```
>>>a = np.array([[1, 2, 3], [4, 5, 6]])
>>>a
array([[1, 2, 3], [4, 5, 6]])

>>>a.shape      # 行数和列数
(2, 3)

>>>a.ndim       # 维数
2

>>>a.size       # 元素数
6
```

改变形状

```
>>>import numpy as np

>>>a = np.arange(0, 20, 1)      # 一维数组

>>>b = a.reshape((4, 5))        # 4行，5列
>>>c = a.reshape((20, 1))       # 2维
>>>d = a.reshape((-1, 4))       # -1: 自动决定行数

>>>a.shape = (4, 5)             # 改变a的形状
```

形状(N,)，(N, 1)和(1, N)不同

形状(N,)：数组是一维的

形状(N, 1)：数组是二维的，N行一列

形状(1, N)：数组是二维的，一行N列

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])     # 一维数组
b = a.copy()

c1 = np.dot(np.transpose(a), b)   # 转置对一维数组不起作用
print(c1)
c2 = np.dot(a, np.transpose(b))   # 转置也可以写成b.T
print(c2)

ax = np.reshape(a, (5, 1))
bx = np.reshape(b, (1, 5))
c = np.dot(ax, bx)
print(c)
```

用相同元素填充数组

```
>>>np.zeros(3)
array([0., 0., 0.])

>>>np.zeros((2, 2), complex)
array([[0.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j]])

>>>np.ones((2, 3))
array([[1., 1., 1.],
       [1., 1., 1.]])

>>>np.full((2,2), 7)
array([[7, 7],
       [7, 7]])
```

用随机数填充数组

rand：0到1之间[0, 1)均匀分布的随机数

```
>>>np.random.rand(2, 4)
array([[ 0.94672374,  0.0383632 ,  0.12738539,  0.21592466],
       [ 0.49394559,  0.2216863 ,  0.3053351 ,  0.51381235]])
```

randn：服从均值为0，方差为1的标准正态（高斯）分布的随机数

```
>>>np.random.randn(2, 4)
array([[ 1.05383548, -1.2142876 , -0.83458293,  0.53291161],
       [ 0.08311765,  0.14007751, -0.06647882,  1.09115942]])
```

也有其他标准概率分布的随机数

一维数组索引与切片

[start : stop]的索引形式可用于从数组中抽取片段（从start位置开始直到stop位置但不包括stop）

```
>>>a = np.array([0, 1, 2, 3, 4])

>>>a[1:3]
array([1, 2])

>>>a[:3]
array([0, 1, 2])

>>>a[1:]
array([1, 2, 3, 4])

>>>a[1:-1]
array([1, 2, 3])
```

一维数组索引与切片

整个数组：a或者a[:]

```
>>>a = np.array([0, 1, 2, 3, 4])
>>>a[:]
array([0, 1, 2, 3, 4])
```

想取出间隔的元素，可以在第二个冒号之后说明第三个数（步长）：

```
>>>a[::2]
array([0, 2, 4])

>>>a[1:4:2]
array([1, 3])
```

步长-1，可用于反转一个数组：

```
>>>a[::-1]
array([4, 3, 2, 1, 0])
```

二维数组索引

多维数组的索引是整数元组：

```
>>>a = np.arange(12); a.shape = (3, 4); a
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])

>>>a[1, 2]
6

>>>a[1, -1]
7
```

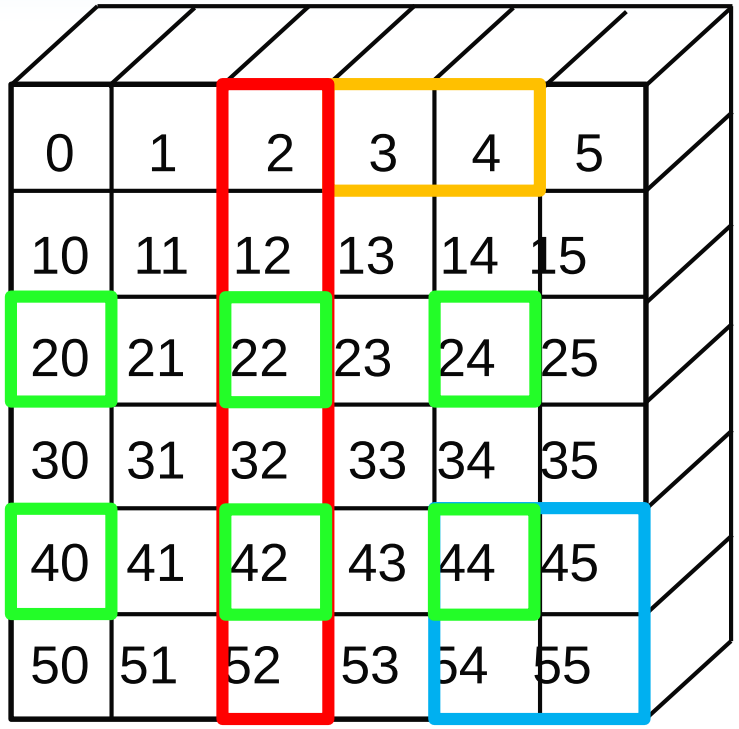
二维数组切片：单行单列

和列表类似

```
>>>a = np.arange(12); a.shape = (3, 4); a
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])

>>>a[:, 1]
array([1, 5, 9])

>>>a[2, :]
array([8, 9, 10, 11])

>>>a[1][2]
6

>>>a[2]
array([8, 9, 10, 11])
```

数组索引

```
>>>a[0, 3:5]
array([3, 4])

>>>a[4:, 4:]
array([[44, 45],
       [54, 55]])

>>>a[:, 2]
array([2, 12, 22, 32, 42, 52])

>>>a[2::2, ::2]
array([[20, 22, 24]
       [40, 42, 44]])
```



拷贝与视图

标准列表的一个切片是它的一个**拷贝**

Numpy数组的一个切片是数组上的一个**视图**，切片数组和原始数组都引用的是同一块内存区域。因而，当改变视图内容时，原始数组的内容也被同样改变了：

```
>>>a = np.arange(5); a
array([0,  1,  2,  3, 4])

>>>b = a[2:]; b
array([2, 3, 4])

>>>b[0] = 100; b
array([100, 3, 4])

>>>a
array([0, 1, 100, 3, 4])
```

拷贝与视图

为了避免改变原数组，可以拷贝切片：

```
>>>a = np.arange(5); a
array([0,  1,  2,  3, 4])

>>>b = a[2:].copy(); b
array([2, 3, 4])

>>>b[0] = 100; b
array([100, 3, 4])

>>>a
array([0, 1, 2, 3, 4])
```

数组计算

基本的算术运算都作用在数组的元素级别

```
import numpy as np

x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)

print(x + y)
print(np.add(x, y))

print(x - y)
print(np.subtract(x, y))

print(x * y)
print(np.multiply(x, y))

print(x / y)
print(np.divide(x, y))

print(np.sqrt(x))
```

矩阵乘法

矩阵乘法是使用dot函数实现的：

```
>>>A = np.array([[1, 2], [3, 4]])
>>>np.dot(A, A)
array([[7, 10],
       [15, 22]])
```

dot函数也可用于矩阵和向量的乘法：

```
>>>A
array([[1, 2],
       [3, 4]])
>>>x = np.array([10, 20])

>>>np.dot(A, x)              #等价于A.dot(x)
array([50, 110])

>>>np.dot(x, A)              #等价于x.dot(A)
array([70, 100])
```

更高效的数学函数

Numpy中包含许多常用的数学函数，例如：np.log, np.maximum, np.sin, np.exp, np.abs等等（详见：https://docs.scipy.org/doc/numpy/reference/routines.math.html ）

大多数情况下，Numpy中的函数比math库中类似的函数更高效，尤其是处理大规模数据时

```
import numpy as np

x = np.array([[1,2],[3,4]])

print(np.sum(x))            # Compute sum of all elements;
print(np.sum(x, axis=0))    # Compute sum of each column;
print(np.sum(x, axis=1))    # Compute sum of each row;
```

保存数组到文件

savetxt()函数将一个数组保存到一个文本文件中：

```
>>>a = np.linspace(0, 1, 12); a.shape = (3, 4); a
array([[ 0.        ,  0.09090909,  0.18181818,  0.27272727],
       [ 0.36363636,  0.45454545,  0.54545455,  0.63636364],
       [ 0.72727273,  0.81818182,  0.90909091,  1.        ]])

>>>np.savetxt("myfile.txt", a)
```

其他格式的文件也可以（参见文档）

save()函数将一个数组存成一个Numpy的“.npy”格式的二进制文件：

```
>>>np.save("myfile", a)
```

生成一个二进制文件myfile.npy包含数组a，之后可以使用np.load()函数读入内存

从文本文件读入数组

loadtxt()函数把一个存成文本文件的数组读入内存

缺省地，该函数假设列是用空白符分隔的。可以通过修改可选的参数来改变此假设。#开头的行被忽略。

示例文本文件data.txt：

```
# Year        Min temp.      Max temp.
  1990            -1.5          25.3
  1991            -3.2          21.2
```

```
>>>table = np.loadtxt("data.txt")
>>>table
array([[1.99000000e+03, -1.50000000e+00, 2.53000000e+01],
       [1.99100000e+03, -3.20000000e+00, 2.12000000e+01]])
```

Matplotlib

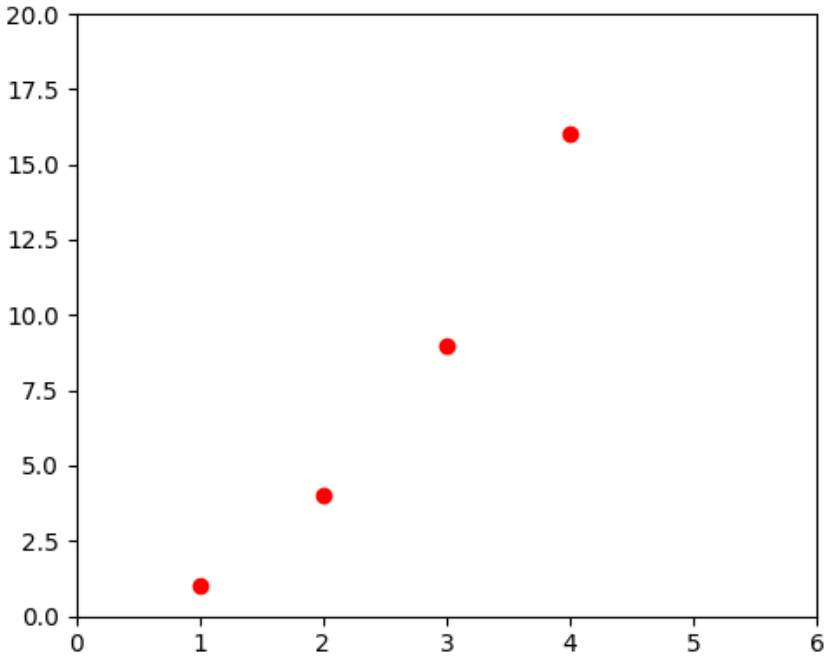
Matplotlib是Python中最常用的可视化工具之一，可以非常方便地创建海量类型的2D图表和一些基本的3D图表

因为在函数的设计上参考了MATLAB，所以叫做Matplotlib

首次发表于2007年，是为了可视化癫痫病人的脑皮层电图相关的信号而研发的，原作者John D. Hunter博士是一名神经生物学家

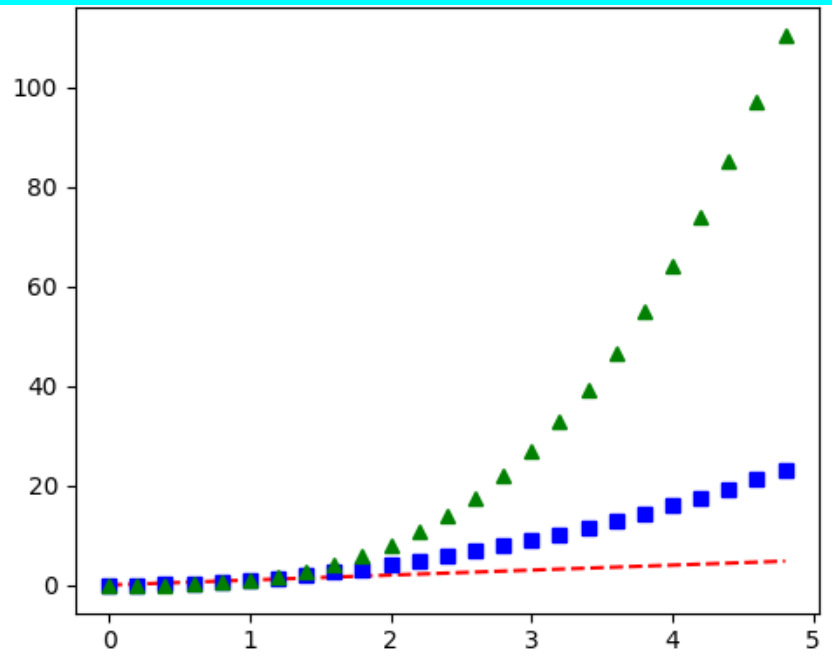
最简单的图表

```
import matplotlib.pyplot as plt

plt.plot([1,2,3,4], [1,4,9,16], 'ro’)

plt.axis([0, 6, 0, 20])

plt.show()
```



一张图表中多个函数（1）

```
import numpy as np
import matplotlib.pyplot as plt

t = np.arange(0., 5., 0.2)
plt.plot(t, t, 'r--', t, t**2, 'bs', t, t**3, 'g^')

plt.show()
```



设置线条属性

使用键值对参数：

```
plt.plot(x, y, linewidth=2.0)
```

使用Line2D类对象的属性设置方法：

```
line, = plt.plot(x, y, '-')

line.set_antialiased(False) # turn off antialiasing
```

使用setp()命令：

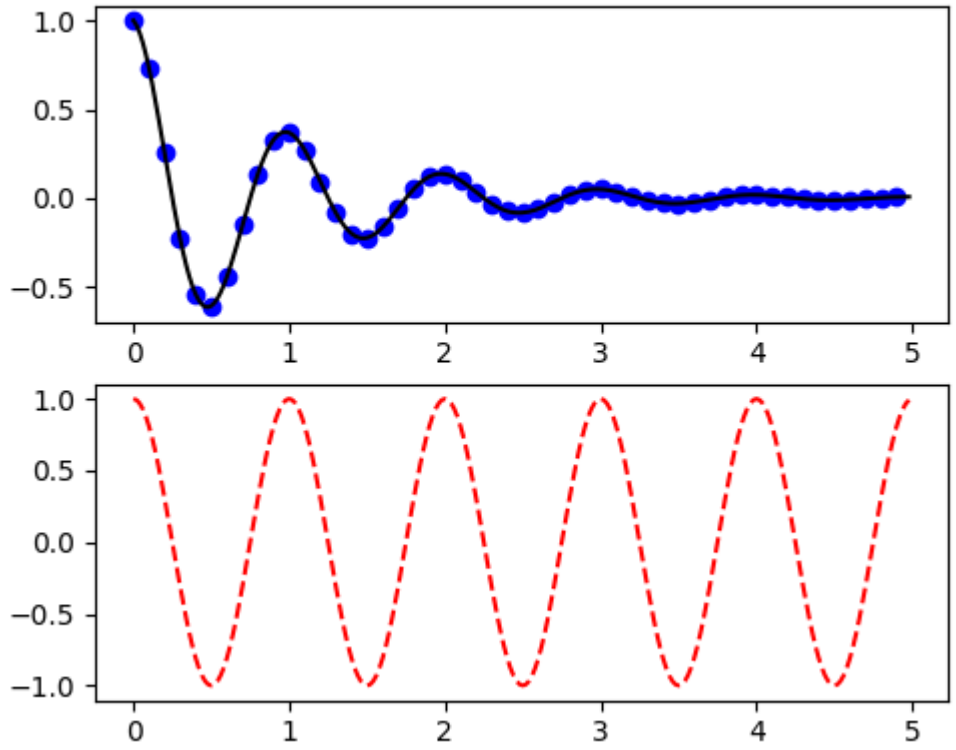
```
lines = plt.plot(x1, y1, x2, y2)

# use keyword args
plt.setp(lines, color='r', linewidth=2.0)
# or MATLAB style string value pairs
plt.setp(lines, 'color', 'r', 'linewidth', 2.0)
```

一张图表中多个函数（2）

```
import numpy as np
import matplotlib.pyplot as plt

# Compute the x and y coordinates for points on sine and
cosine curves
x = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x)
y_cos = np.cos(x)

# Plot the points using matplotlib
plt.plot(x, y_sin)
plt.plot(x, y_cos)

plt.xlabel('x axis label')
plt.ylabel('y axis label')
plt.title('Sine and Cosine')
plt.legend(['Sine', 'Cosine’])

plt.show()
```

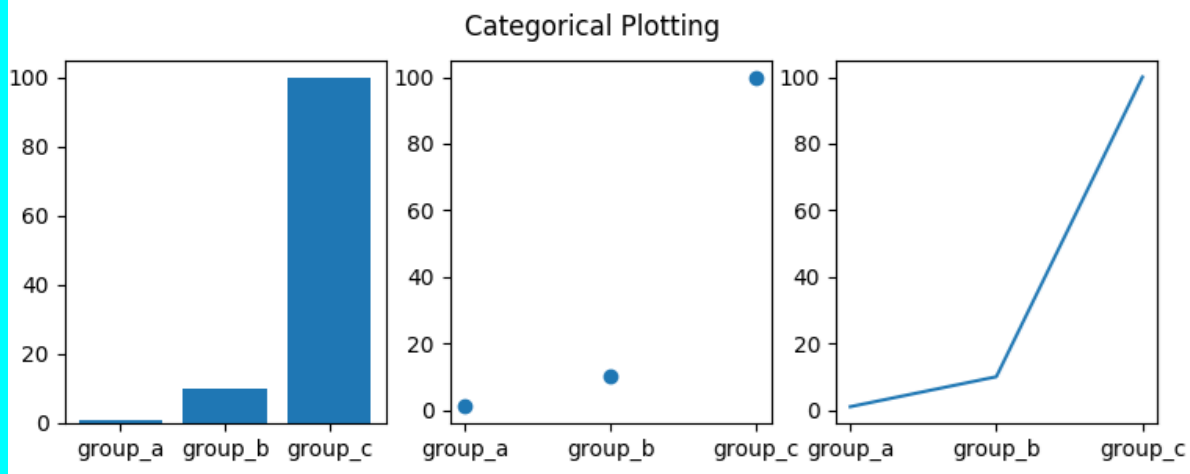
多张图表：子图表

```
import numpy as np
import matplotlib.pyplot as plt

def f(t):
    return np.exp(-t) * np.cos(2*np.pi*t)

t1 = np.arange(0.0, 5.0, 0.1)
t2 = np.arange(0.0, 5.0, 0.02)

plt.figure(1)
plt.subplot(211)
plt.plot(t1, f(t1), 'bo', t2, f(t2), 'k')

plt.subplot(212)
plt.plot(t2, np.cos(2*np.pi*t2), 'r--')

plt.show()
```



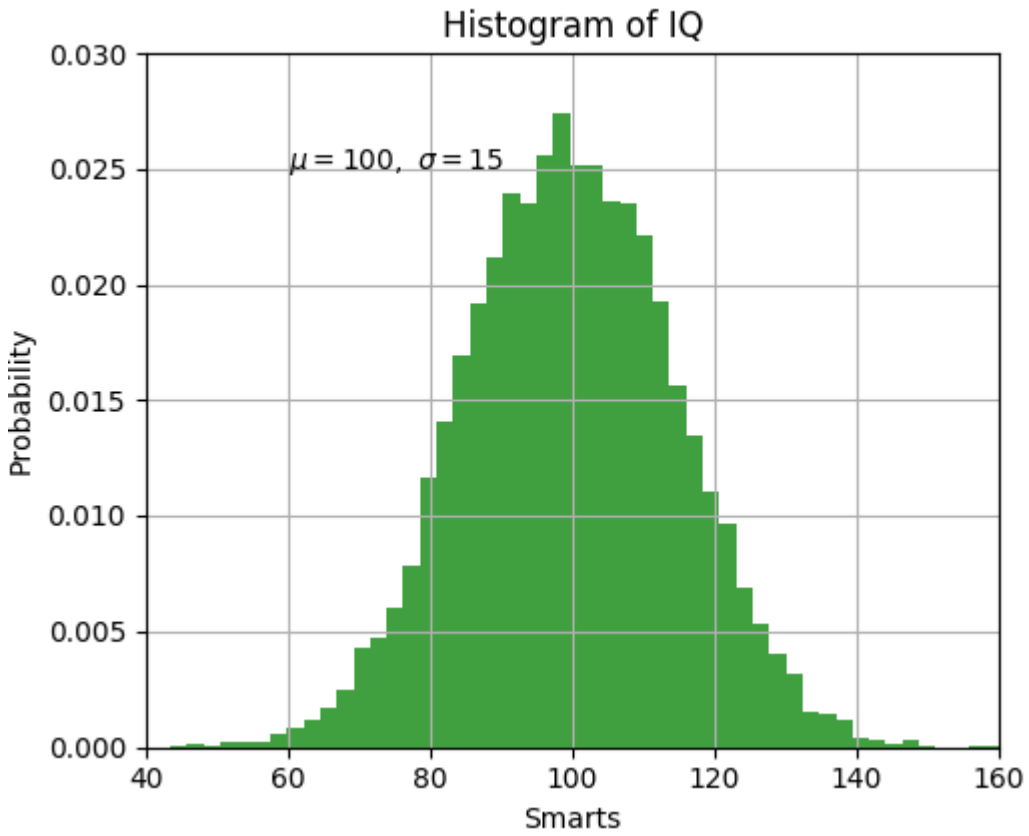
绘制分类变量的图表

```
names = ['group_a', 'group_b', 'group_c']
values = [1, 10, 100]

plt.figure(1, figsize=(9, 3))

plt.subplot(131)
plt.bar(names, values)

plt.subplot(132)
plt.scatter(names, values)

plt.subplot(133)
plt.plot(names, values)

plt.suptitle('Categorical Plotting’)

plt.show()
```



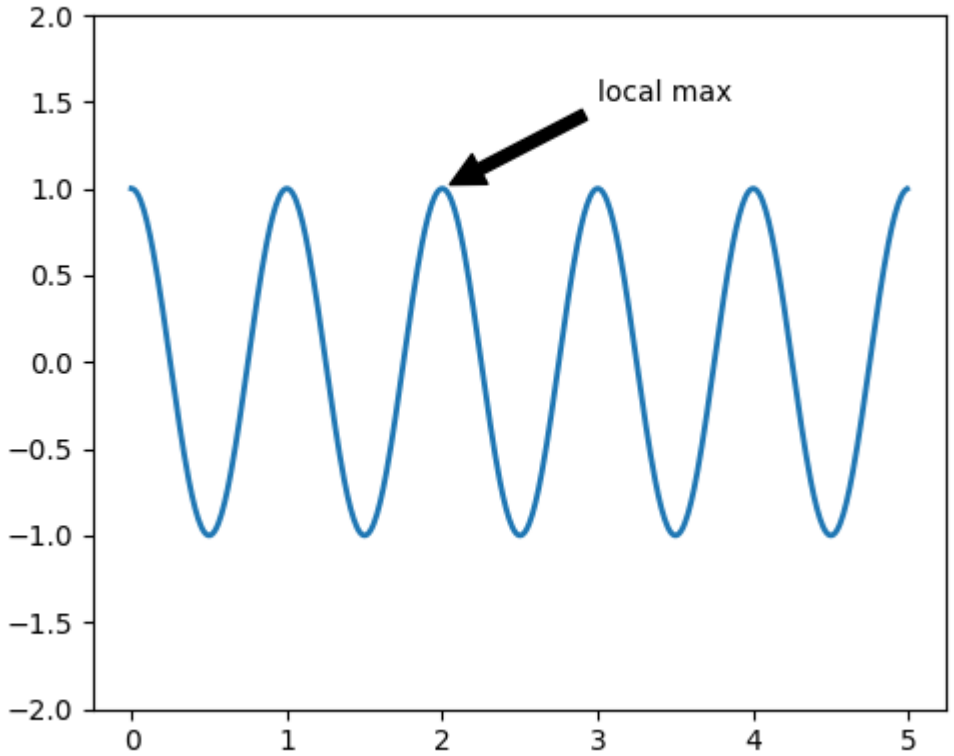
添加文本

```
import numpy as np
import matplotlib.pyplot as plt

mu, sigma = 100, 15
x = mu + sigma * np.random.randn(10000)

# the histogram of the data
n, bins, patches = plt.hist(x, 50, density=1,
facecolor='g', alpha=0.75)

plt.xlabel('Smarts')
plt.ylabel('Probability')
plt.title('Histogram of IQ')
plt.text(60, .025, r'$\mu=100,\ \sigma=15$')
plt.axis([40, 160, 0, 0.03])
plt.grid(True)

plt.show()
```



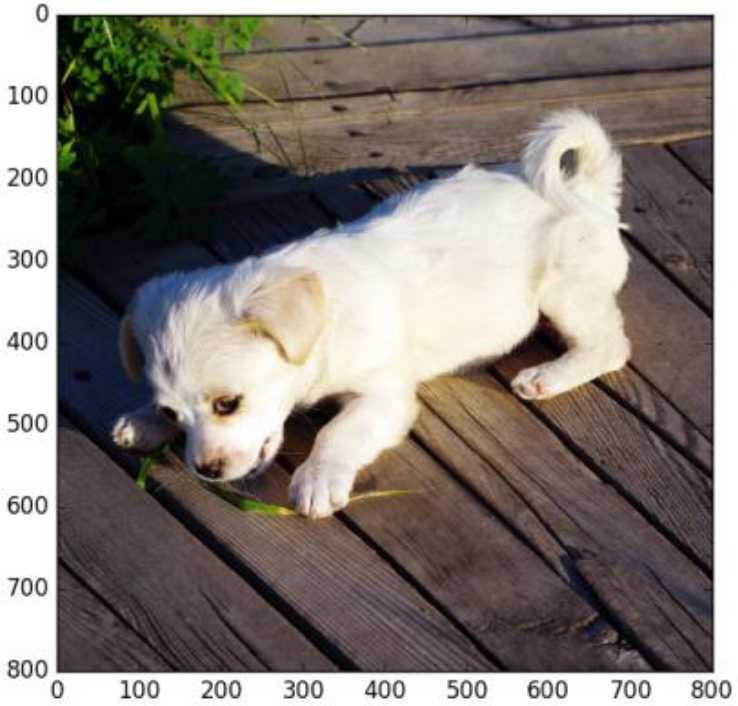
添加文本注释

```
import numpy as np
import matplotlib.pyplot as plt

ax = plt.subplot(111)

t = np.arange(0.0, 5.0, 0.01)
s = np.cos(2*np.pi*t)

line, = plt.plot(t, s, lw=2)

plt.annotate('local max', xy=(2, 1),
             xytext=(3, 1.5),
           arrowprops=dict(facecolor='black',
             shrink=0.05))
plt.ylim(-2,2)

plt.show()
```



图像显示

```
import matplotlib.pyplot as plt

plt.figure('A Little White Dog')
little_dog_img = plt.imread('little_white_dog.jpg')
plt.imshow(little_dog_img)
Plt.show()
```



Pandas

Pandas是python的一个数据分析包

由AQR Capital Management于2008年4月开发，并于2009年底开源出来

导入惯例：

```
>>>from pandas import Series, DataFrame
>>>import pandas as pd
```

因为Series和DataFrame用的次数非常多，所以将其引入本地命名空间中会更方便

常用数据结构

1. Series

- 一维**标记**数组，由一组**数据**（各种NumPy数据类型）以及一组与之相关的**数据标签**（即索引）组成。
- 类似于Numpy中的一维数组和Python的列表，不同之处是数组和series中存放的是**相同类型**的元素

2. DataFrame

- 二维表格型数据结构， 含有一组有序的列，每列可以是不同的值类型（数值、字符串、布尔值等），
- 每列都有标签，可看成一个**Series的字典**

3. Panel

- 三维数组，可以理解为DataFrame的容器
- Panel data源于经济学，也是pan(el)-da(ta)-s的名字来源

创建Series：传入列表

默认整型索引

```
>>>obj = Series([4, 7, -5, 3])
>>>obj
0    4
1    7
2   -5
3    3
dtype: int64

>>>obj.values
array([4, 7, -5, 3], dtype=int64)

>>>obj.index
RangeIndex(start=0, stop=4, step=1)
```

创建Series：传入列表

给定索引

```
>>>obj2 = Series([4,7,-5,3], index=['d','b','a','c'])
>>>obj2
d    4
b    7
a   -5
c    3
dtype: int64

>>> obj2.index
Index(['d', 'b', 'a', 'c'], dtype='object')
```

访问Series中的元素

可以使用索引来选取Series中的单个或一组值

```
>>>obj2['a']
-5

>>>obj2['d']= 6

>>>obj2[['c','a','d']]
c     3
a    -5
d     6
dtype: int64
```

对Series的操作

NumPy**数组操作**，如通过一个布尔数组过滤，纯量乘法，或使用数学函数，都会保持索引和值间的关联：

```
>>>obj2[obj2 > 0]
d    4
b    7
c    3
dtype: int64

>>>obj2*2
d     8
b    14
a   -10
c     6
dtype: int64

>>>np.exp(obj2)
d      54.598150
b    1096.633158
a       0.006738
c      20.085537
dtype: float64
```

对Series的操作

还可将Series看成是一个**定长的有序字典**，因为它是索引值到数据值的一个映射。它可以用在许多原本需要字典参数的函数中：

```
>>>'b' in obj2
True

>>>'e' in obj2
False
```

创建Series：传入字典

如果数据被存放在一个Python字典中，也可以直接通过这个字典来创建Series
如果只传入一个字典，则结果Series中的索引就是原字典的键（有序排列）

```
>>>sdata = {'Ohio': 35000, 'Texas': 71000, 'Oregon':
16000, 'Utah': 5000}
>>>obj3 = Series(sdata)
>>>obj3
Ohio      35000
Oregon    16000
Texas     71000
Utah       5000
dtype: int64
```

创建Series：传入字典

下例中，sdata跟states索引相匹配的那3个值会被找出来并放到相应的位置上，但由于“California”所对应的sdata值找不到，所以其结果就为NaN（Not A Number，非数字）

```
>>>states = ['California', 'Ohio', 'Oregon', 'Texas']
>>>obj4 = Series(sdata, index=states)
>>>obj4
California      NaN
Ohio          35000
Oregon        16000
Texas         71000
dtype: float64
```

检测缺失数据

pandas的**isnull**和**notnull**函数可用于检测缺失数据

Series也提供了类似的实例方法，如obj4.isnull()

```
>>>pd.isnull(obj4)
California      True
Ohio           False
Oregon         False
Texas          False
dtype: bool

>>>pd.notnull(obj4)
California     False
Ohio            True
Oregon          True
Texas           True
dtype: bool
```

自动对齐索引

Series在**算术运算**中会自动对齐不同索引的数据

```
>>>obj3
Ohio      35000
Oregon    16000
Texas     71000
Utah       5000
dtype: int64
>>>obj4
California      NaN
Ohio          35000
Oregon        16000
Texas         71000
dtype: float64
>>>obj3 + obj4
California       NaN
Ohio           70000
Oregon         32000
Texas         142000
Utah             NaN
dtype: float64
```

Series对象及其索引的name

```
>>>obj4.name = 'population'

>>>obj4.index.name = 'state'

>>>obj4
state
California      NaN
Ohio          35000
Oregon        16000
Texas         71000
Name: population, dtype: float64
```

修改索引

```
>>>obj
0     4
1     7
2    -5
3     3

>>>obj.index = ['Bob', 'Steve', 'Jeff', 'Ryan']

>>>obj
Bob       4
Steve     7
Jeff     -5
Ryan      3
dtype: int64
```

创建DataFrame（1）

DataFrame既有行索引也有列索引，它可以被看做**由Series组成的字典**（共用同一个索引）。

最常用的创建方法是直接传入一个由等长列表或NumPy数组构成的字典

```
>>>data={'state':['Ohio','Ohio','Ohio','Nevada','Nevada'],
      'year':[2000, 2001, 2002, 2001, 2002],
      'pop':[1.5, 1.7, 3.6, 2.4, 2.9]}
>>>frame = DataFrame(data)
>>>frame
   pop   state  year
0  1.5    Ohio  2000
1  1.7    Ohio  2001
2  3.6    Ohio  2002
3  2.4  Nevada  2001
4  2.9  Nevada  2002
```

创建DataFrame（1）

如果指定了列序列，DataFrame的列就会按指定顺序排列

```
>>>DataFrame(data, columns=['year', 'state', 'pop'])
```

跟Series一样，如果传入的列在数据中找不到，就会产生NaN值

```
>>>frame2=DataFrame(data, columns=['year', 'state', 'pop', 'debt'],
            index=['one', 'two', 'three', 'four', 'five'])
>>>frame2
       year   state  pop debt
one    2000    Ohio  1.5  NaN
two    2001    Ohio  1.7  NaN
three  2002    Ohio  3.6  NaN
four   2001  Nevada  2.4  NaN
five   2002  Nevada  2.9  NaN
```

访问单列

通过字典记法或属性，可以将DataFrame的列获取为一个Series:

```
>>>frame2['state']
one         Ohio
two         Ohio
three       Ohio
four      Nevada
five      Nevada
Name: state, dtype: object

>>>frame2.year
one        2000
two       2001
three      2002
four       2001
five       2002
Name: year, dtype: int64
```

访问单行

行也可以使用一些方法通过位置（iloc）或名字（loc）来检索

```
>>>frame2.loc['three']
year      2002
state     Ohio
pop        3.6
debt       NaN
Name: three, dtype: object

>>> frame2.iloc[2]
Out[15]:
year      2002
state     Ohio
pop        3.6
debt       NaN
Name: three, dtype: object
```

修改列

```
>>>frame2['debt'] = 16.5
>>>frame2
       year   state  pop  debt
one    2000    Ohio  1.5  16.5
two    2001    Ohio  1.7  16.5
three  2002    Ohio  3.6  16.5
four   2001  Nevada  2.4  16.5
five   2002  Nevada  2.9  16.5

>>>frame2['debt'] = np.arange(5)
>>>frame2
       year   state  pop  debt
one    2000    Ohio  1.5     0
two    2001    Ohio  1.7     1
three  2002    Ohio  3.6     2
four   2001  Nevada  2.4     3
five   2002  Nevada  2.9     4
```

修改列

```
>>>val = Series([-1.2, -1.5, -1.7], index=[ 'two', 'four',
'five'])
>>>frame2['debt'] = val
>>>frame2
       year   state  pop  debt
one    2000    Ohio  1.5   NaN
two    2001    Ohio  1.7  -1.2
three  2002    Ohio  3.6   NaN
four   2001  Nevada  2.4  -1.5
five   2002  Nevada  2.9  -1.7
```

增加列和删除列

```
>>>frame2['eastern'] = frame2.state == 'Ohio'
>>>frame2
       year   state  pop  debt  eastern
one    2000    Ohio  1.5   NaN     True
two    2001    Ohio  1.7  -1.2     True
three  2002    Ohio  3.6   NaN     True
four   2001  Nevada  2.4  -1.5    False
five   2002  Nevada  2.9  -1.7    False

>>>del frame2['eastern']
>>>frame2
       year   state  pop  debt
one    2000    Ohio  1.5   NaN
two    2001    Ohio  1.7  -1.2
three  2002    Ohio  3.6   NaN
four   2001  Nevada  2.4  -1.5
five   2002  Nevada  2.9  -1.7
```

删除行或列

```
>>> frame2.drop(['pop','debt'], axis=1)        # 删除pop和debt列
       year   state
one    2000    Ohio
two    2001    Ohio
three  2002    Ohio
four   2001  Nevada
five   2002  Nevada

>>> frame2.drop(columns=['pop','debt’])        # 删除pop和debt列
       year   state
one    2000    Ohio
two    2001    Ohio
three  2002    Ohio
four   2001  Nevada
five   2002  Nevada

>>> frame2.drop(['one', 'three', 'five'], axis=0) # 删除one, three, five行
       year   state  pop  debt
two    2001    Ohio  1.7  -1.2
four   2001  Nevada  2.4  -1.5

>>>frame2.drop([‘pop’,‘debt’], axis=1, inplace=True)
```

创建DataFrame（2）

传入嵌套字典（字典的字典），外部键会被解释为列索引，内部键会被解释为行索引：

```
>>>pop = {'Nevada': {2001: 2.4, 2002: 2.9},
        'Ohio': {2000: 1.5, 2001: 1.7, 2002:3.6}}
>>>frame3 = DataFrame(pop)
>>>frame3
       Nevada  Ohio
2000      NaN   1.5
2001      2.4   1.7
2002      2.9   3.6
```

```
>>>frame4 = DataFrame(pop, index=[2001, 2002, 2003])
>>>frame4
       Nevada  Ohio
2001      2.4   1.7
2002      2.9   3.6
2003      NaN   NaN
```

缺失数据处理

删除任何有缺失数据的行：

```
>>>frame3.dropna(how='any')
       Nevada  Ohio
2001     2.4   1.7
2002     2.9   3.6
```

对缺失值进行填充：

```
>>>frame3.fillna(value=5)
       Nevada  Ohio
2000     5.0   1.5
2001     2.4   1.7
2002     2.9   3.6
```

判断哪些值是缺失值（nan）：

```
>>>pd.isna(frame3)
       Nevada  Ohio
2000    True   False
2001    False  False
2002    False  False
```

查看数据

查看DataFrame前n行或后n行：

frame.head(3)；frame.tail(3)

查看DataFrame的索引、列以及底层的Numpy数据：

frame.index ；frame.columns ；frame.values

显示数据的快速统计汇总：

frame.describe() 对每一列数据进行统计，包括计数、均值、标准差、各个分位数等

转置数据：

frame.T

对轴排序：

frame.sort_index(axis=1, ascending=False)，其中axis=1表示对所有的列索引进行排序，下面的数也跟着发生移动。

对值排序：

frame.sort_values(by=‘Name’) 对name这一列，从小到大进行排序

选择行与列

选取多行或多列：

frame[[‘state’ , ‘pop’]]，选择’ state’ 和’ pop’ 两列，结果是一个DataFrame

frame[0:3]，选择前三行

loc用标签选择数据：

frame.loc[‘one’]，选择索引为’one’的行

frame.loc[‘one’ , ‘pop’]，选择 ‘one’ 行，’ pop’ 列

frame.loc[:, [‘state’ , ‘pop’]]，选择所有行，’ state’ 和’ pop’ 列

frame.loc[[‘one’ , ‘two’], [‘state’ , ‘pop’]]，选择’ one’ 和’ two’ 行，’ state’ 和’ pop’ 列

iloc用位置选择数据：

frame.iloc[1:2, 1:2]

frame.iloc[[0,2], [1,2]]

使用条件来选择：

frame[frame.year>2001]，选择year列中大于2001的数据

frame[frame>2001]，选择frame中所有大于2001的数据

frame[frame[‘year’].isin([‘2000’ , ‘2002’])]，选择year列的值为’ 2000’ ,’ 2002’ 的所有行

相关操作

统计数据：

a.mean()，对a的每一列数据值求平均值；

a.mean(1)，对a的每一行数据值求平均值

a[‘x’].value_counts()，统计列x中各值出现的次数

对数据应用函数：

a.apply(lambda x : x.max() - x.min())，对a的每一列，返回最大值和最小值的差

字符串操作：

a[‘gender1’].str.lower()，将gender1中所有的英文转化为小写，注意Dataframe没有str属性，只有Series有，所以要选取a中的gender1列。

读取与写入文件

写入.csv文件：

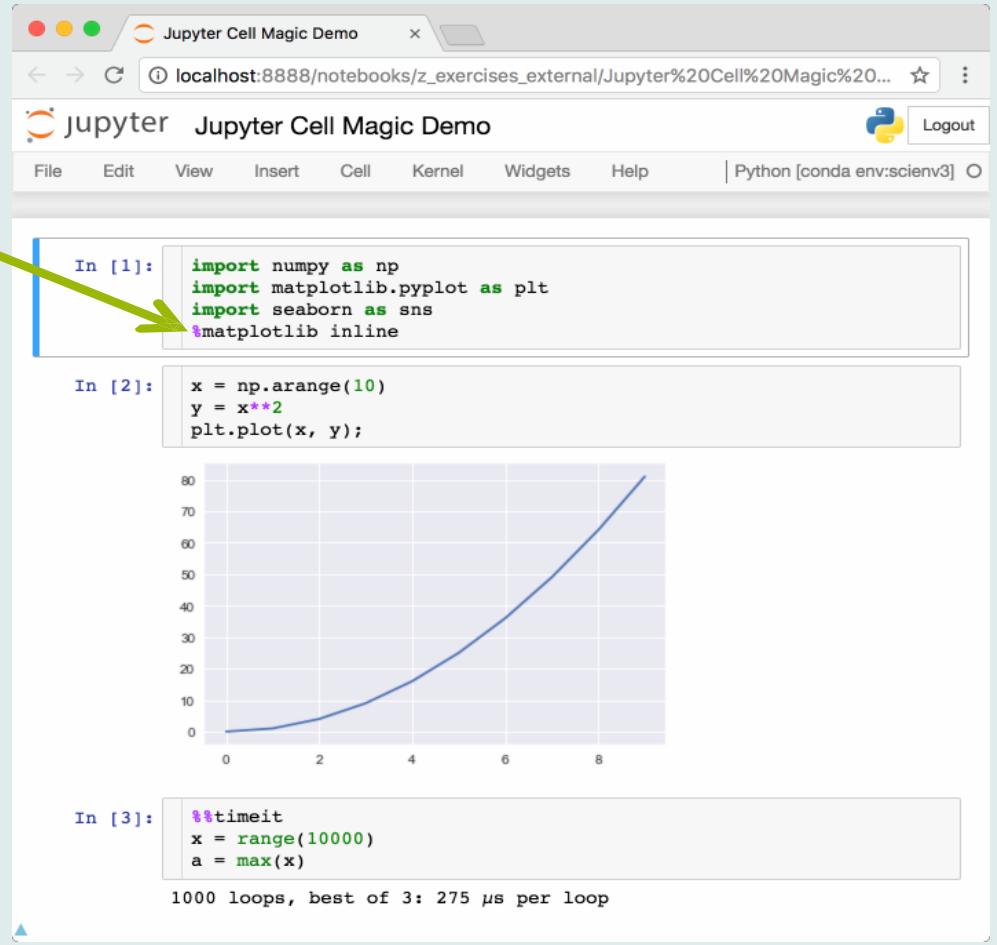
```
frame3.to_csv( ‘C:\\Users\\qiuyu\\frame3.csv’ )
```

读取.csv文件：

```
frame4 = pd.read_csv('C:\\Users\\qiuyu\\frame3.csv')
frame4 = pd.read_csv('C:\\Users\\qiuyu\\frame3.csv',
index_col=0)
```

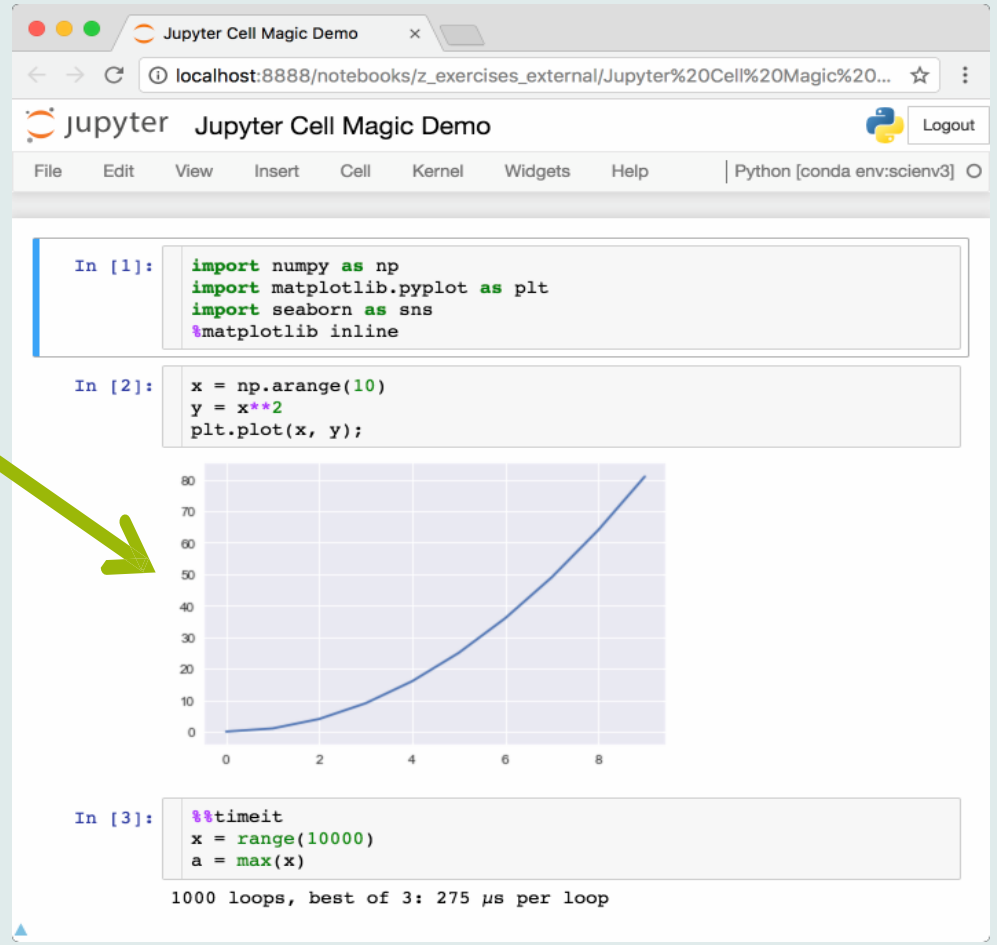
Jupyter魔术命令

- %matplotlib inline
  - 将图表显示在Jupyter notebook中



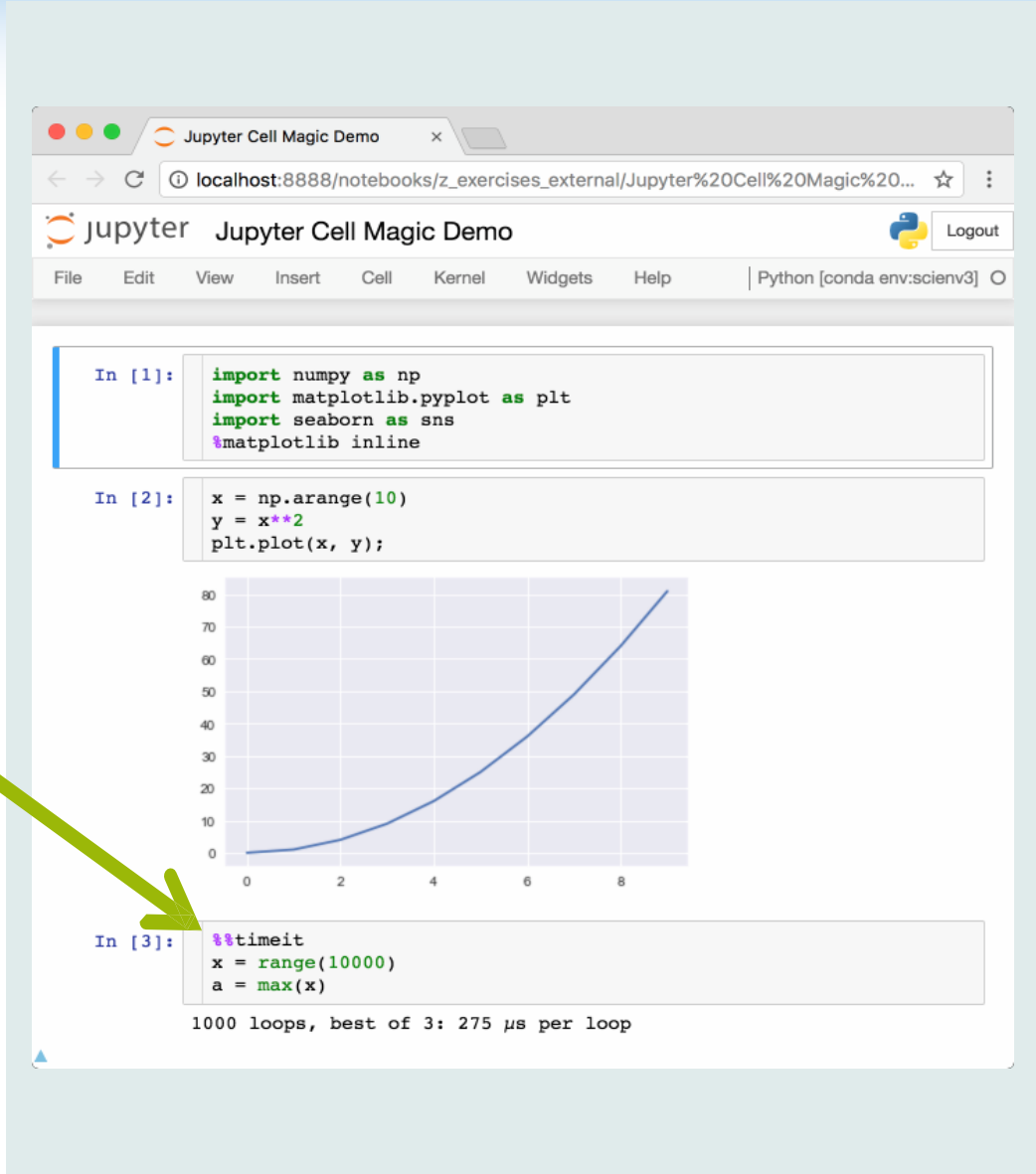
Jupyter魔术命令

- %matplotlib inline
  - 将图表显示在Jupyter notebook中



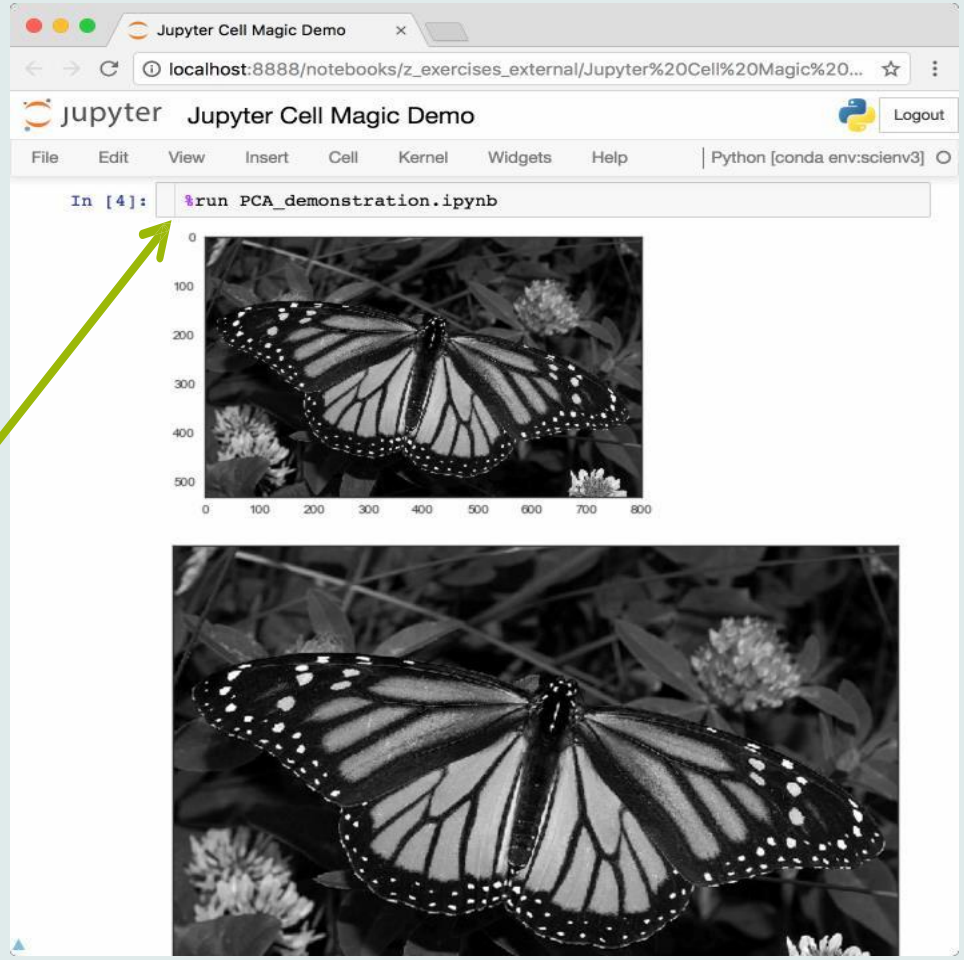
Jupyter魔术命令

- %matplotlib inline
  - 将图表显示在Jupyter notebook中
- %%timeit
  - 记录一个单元的执行时间



Jupyter魔术命令

- %matplotlib inline
  - 将图表显示在Jupyter notebook中
- %%timeit
  - 记录一个单元的执行时间
- %run filename.ipynb
  - 运行另一个notebook或python文件的代码

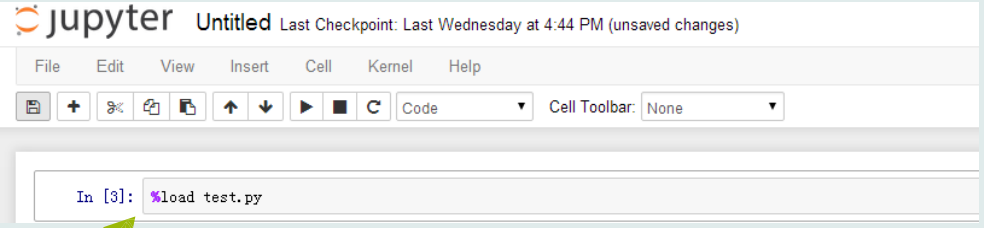


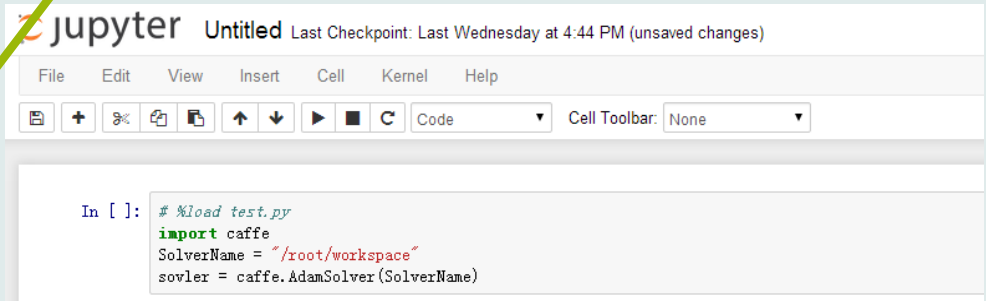
Jupyter魔术命令

- %matplotlib inline
  - 将图表显示在Jupyter notebook中
- %%timeit
  - 记录一个单元的执行时间
- %run filename.ipynb
  - 运行另一个notebook或python文件的代码
- %load filename.py
  - 将外部文件的内容拷贝粘贴到单元中

test.py

```
import caffe
SolverName = "/root/workspace“
sovler = caffe.AdamSolver(SolverName)
```





Jupyter快捷键

Keyboard shortcuts

The Jupyter Notebook has two different keyboard input modes. **Edit mode** allows you to type code/text into a cell and is indicated by a green cell border. **Command mode** binds the keyboard to notebook level actions and is indicated by a grey cell border with a blue left margin.

Command Mode (press Esc to enable)

F : find and replace	Shift-J : extend selected cells below
Ctrl-Shift-P : open the command palette	A : insert cell above
Enter : enter edit mode	B : insert cell below
Shift-Enter : run cell, select below	X : cut selected cells
Ctrl-Enter : run selected cells	C : copy selected cells
Alt-Enter : run cell, insert below	Shift-V : paste cells above

可以通过Help → Keyboard Shortcuts查看所有的快捷键

RUNOOB.COM

shell

首页 菜鸟工具 菜鸟笔记 参考手册 用户笔记 测验/考试 本地书签

全部教程

- HTML / CSS
- JavaScript
- 服务端
- 数据库
- AI & 数据分析
- 移动端
- 开发工具
- XML 教程
- ASP.NET
- Web Service
- 网站建设

AI & 数据分析

【学习 Python】
Python3.x 版本，未来主流版本。

【学习 NumPy】
NumPy 是 Python 语言的一个扩展程序库，支持大量的维度数组与矩阵运算

【学习 Pandas】
Pandas 是 Python 语言的一个扩展程序库，用于数据分析

【学习 Matplotlib】
Matplotlib 是 Python 的绘图库

【学习 Scipy】
SciPy 是一个开源的 Python 算法库和数学工具包。

【学习 Pytorch】
Pytorch 是一个开源的机器学习库。

【学习 Ollama】
开源的本地大语言模型运行框架。

【机器学习】
机器学习是让机器通过经验（数据）来做决策和预测。

【学习 scikit-learn】
scikit-learn 是一个开源的机器学习库。

【学习 R】
R 语言是为数学研究工作者设计的一种数学编程语言

【学习 OpenCV】
开源的计算机视觉和机器学习软件库

【学习 Selenium】
用于自动化 Web 浏览器操作的工具集

【学习 Julia】
Julia 是一个面向科学计算的高性能动态高级程序设计语言

【量化交易】
利用计算机编程来进行金融交易的一种策略。

【学习 Dash】
构建数据分析和数据可视化的 Web 应用程序。